

Evaluation of antivirus and endpoint detection systems using user-land function hooking as a behavioural, real time defence technique

Lee Dale

Submitted as fulfilment of the requirements of Edinburgh Napier University for the degree of Master of Science in Cyber Security

School of Computing
December 2024

Supervisor Richard Macfarlane

MSc dissertation checklist

Milestones	Date of completion	Target deadline
Proposal		Week 3
Initial report		Week 7
Full draft of the dissertation		2 weeks before final deadline

Learning outcome	The markers will assess	Pages[1]	Hours spent
Learning outcome 1 Conduct a literature search using an appropriate range of information sources and produce a critical review of the findings.	* Range of materials; list of references * The literature review/exposition/background information chapter		(for example, 200 hours)
Learning outcome 2 Demonstrate professional competence by sound project management and (a) by applying appropriate theoretical and practical computing concepts and techniques to a non-trivial problem, or (b) by undertaking an approved project of equivalent standard.	* Evidence of project management (Gantt chart, diary, etc.) * Depending on the topic: chapters on design, implementation, methods, experiments, results, etc.		(for example, 200 hours)
Learning outcome 3 Show a capacity for self-appraisal by analysing the strengths and weakness of the project outcomes with reference to the initial objectives, and to the work of others.	* Chapter on evaluation (assessing your outcomes against the project aims and objectives) * Discussion of your project's output compared to the work of others.		

1 Please note the page numbers where evidence of meeting the learning outcome can be found in your dissertation.

			(for example, 100 hours)
Learning outcome 4 Provide evidence of the meeting learning outcomes 1-3 in the form of a dissertation which complies with the requirements of the School of Computing both in style and content.	* Is the dissertation well-written (academic writing style, grammatical), spell-checked, free of typos, neatly formatted. * Does the dissertation contain all relevant chapters, appendices, title and contents pages, etc. * Style and content of the dissertation.		(for example, 80 hours)
Learning outcome 5 Defend the work orally at a viva voce examination.	* Performance * Confirm authorship		1 hour

Have you previously uploaded your dissertation to Turnitin? Yes/No

Has your supervisor seen a full draft of the dissertation before submission? Yes/No

Has your supervisor said that you are ready to submit the dissertation? Yes/No

Authorship Declaration

I, <insert your full name>, confirm that this dissertation and the work presented in it are my own achievement.

Where I have consulted the published work of others this is always clearly attributed;

Where I have quoted from the work of others the source is always given. With the exception of such quotations this dissertation is entirely my own work;

I have acknowledged all main sources of help;

If my research follows on from previous work or is part of a larger collaborative research project I have made clear exactly what was done by others and what I have contributed myself;

I have read and understand the penalties associated with Academic Misconduct.

I also confirm that I have obtained **informed consent** from all people I have involved in the work in this dissertation following the School's ethical guidelines

Signed:

Date: 15. November 2024

Matriculation no: 40615561

General Data Protection Regulation Declaration

Under the General Data Protection Regulation (GDPR) (EU) 2016/679, the University cannot disclose your grade to an unauthorised person. However, other students benefit from studying dissertations that have their grades attached.

Please sign your name below *one* of the options below to state your preference.

The University may make this dissertation, with indicative grade, available to others.

The University may make this dissertation available to others, but the grade may not be disclosed.

The University may not make this dissertation available to others.

Abstract

Contents

Evaluation of antivirus and endpoint detection systems using user-land function hooking as a behavioural, real time defence technique 1

Submitted as fulfilment of the requirements of Edinburgh Napier University for the degree of Master of Science in Cyber Security 1

School of Computing....1

December 2024....1

Abstract....6

1 Introduction....9

1.1 Background....9

2 Aims and Objectives 10

3 Literature Review.... 10

3.1 What is Ransomware? 10

3.1 Landscape 12

3.1 Stages of a ransomware attack.... 12

3.3 Characteristics of cryptographic ransomware 14

3.3 Tactics and Techniques employed by current ransomware 16

Execution 16

Persistence.... 17

Defence Evasion 17

Lateral movement.... 17

Command and control 17

Impact.... 18

Data Exfiltration.... 18

Encryption.... 18

3.2 Detection techniques used by AV/EDR Systems.... 19

3.2 Detection timing 21

3.2 What is function hooking? 21

3.2 Endpoint Detection Function hooking.... 22

3.4 Windows API function calls vs syscalls 24

Types of function hooking 26

Userland hooks 26

Hooking example.... 27

Kernel level hooks.... 29

3.8 Detection based on function hooking.... 29

3.6 Evasion techniques 31

API Un-hooking 31

Direct syscall instantiation 33

Indirect syscall instantiation 33
Process Injection 34
3.2 Ransomware evasion 34
Conclusions 35
6 Research Methodology Design 36
6.1 Research Questions 37
6.2 Chosen AV/EDR tools 37
6.1 Research Methodology 38
Evaluation of results 39
Tools used 40
6.3 Experimental Environment Setup 40
Host Machine 40
Virtual Machine 40
6.2 Experiments and Results 41
RQ1 41
Result 43
RQ2 45
Baseline experiment 45
Results 45
Indirect syscall experiments 47
Results 47
Sophos 49
OpenEDR 49
Cisco Secure Endpoint 50
Discussion of Results 53
Aims and Objectives 54
References 56
Appendix 61

1 Introduction

The aim of this work is to first understand the current research on ransomware in relation to the stages of attack and the attack tactics and techniques used. Secondly, to focus on evasion tactics employed by ransomware to avoid being detected by antivirus and endpoint detection systems installed on host machines. The particular focus on this work is around the hooking of API function calls in the Windows operating system which is employed by current antivirus software for use in detection and prevention of ransomware techniques.

Most modern AV/EDR systems use the technique of Windows API function hooking in order to intercept process functionality for use in malicious code detection. The heuristics used in determining ransomware behaviour are unique to each AV/EDR system, however, most systems use some kind of machine learning classifier to distinguish between malicious and benign applications. Before detection systems can execute ML algorithms, they need to intercept process behaviour to insert themselves between the process and the operating system. AV/EDR systems generally employ user land hooks to do this, and many systems hook the lower-level native functions to allow them to intercept more functionality with less performance overhead.

However, there are various techniques that can be used to avoid these user land hooks, and this work identifies the techniques used and determines which current AV/EDR systems are vulnerable to the most current technique. Some other heuristics may be used in proprietary systems by the AV/EDR system alongside API hooking to determine malicious behaviour, but this is beyond the scope of this work.

This work will focus on the techniques identified in the literature that can be used to evade function hooking employed by AV/EDR systems and evaluates whether these techniques are still valid and result in successful evasion. If malicious code is able to disable or circumvent the AV/EDR function hooks, then a major defence layer for detection systems is taken away.

The concept of function hooking for defence and subsequent evasion techniques have been around for years and this is reflected in the literature. However, specific techniques such as indirect syscalls are relatively new and are underrepresented in the literature. On top of this, there is a lack of analysis around understanding whether AV/EDR tools still employ function hooking in their defence layers and if so to what extent.

AV/EDR tools are constantly changing their defence techniques and because commercial tools tend to not publicly disclose their internal workings, there is a lack of understanding on which techniques are still employed, specifically around function hooking. Ransomware is also constantly evolving its evasion techniques and AV/EDR vendors need to identify the gaps in their defence layers caused by these evasion techniques.

The output of this work will be twofold: first it is established which hooks are employed by AV/EDR systems if at all. This will determine whether AV/EDR vendors are still employing function hooking as a defence technique. It will also determine whether the evasion techniques identified from the literature are viable against those AV/EDR systems. Experiments are then carried out on each AV/EDR system using tools that employ the evasion techniques discussed. This attempts to answer the question of whether Windows API function hooks can be evaded and AV/EDR system defences can be bypassed for the systems tested.

1.1 Background

As of 2021, ransomware was designated as the leading type of cyber-attack and still holds that title today. In recent years it has become apparent that more and more of our daily lives are coupled to digital services and these digital services are becoming prevalent in critical systems such as health care, military and financial services. Recently there has been an effort by

ransomware gangs to target these critical systems around the world. In the UK there has been a recent attack on the NHS which is alleged to be carried out by the ransomware group BlackBasta. This echoes the 2017 attack on the NHS by the WannaCry ransomware which according to the Nation Audit Office in the UK affected more than a third of NHS trusts and resulted in at least 6,900 hospital appointments cancelled. Globally the average cost of a ransomware incident was said to be around USD 1.85 million. With all this in mind, prevention of ransomware from affecting computer systems in any industry is a lively research area.

The main tactic of ransomware is to encrypt files on a computer with a key only known to the attackers carrying out the ransomware campaign. The idea is to prevent legitimate users from accessing files on their system in order to extort payments from them.

Ransomware has become such big business now that modern variants have been turned into Ransomware as a Service, where the developers of the ransomware are not the ones carrying out the attacks. The developers of the ransomware create a turn-key solution to sell to others in which they take a percentage cut from the extortion payments.

2 Aims and Objectives

The aim of this work is to build on previous studies [1] [2] that have shown that anti-virus and endpoint detection and response systems are vulnerable to function hooking evasion. This work outlines the specific evasion techniques used to deliver ransomware payloads by avoiding the function hooking layer of AV/EDR systems. This work also determines whether these tactics are still valid.

The aims below support this goal:

Aim one: Carry out literature review of ransomware behaviour and the threat landscape related to AV/EDR evasion, focusing on Windows API function hooking as a defence technique.

Aim two: From the results of the literature review, evaluate function hooking in current AV/EDR systems to determine the extent of use in each tool.

Aim three: Carry out experiments to evaluate the level of defence in each AV/EDR tool when it comes to detection of malicious payloads.

Aim Four: Provide analysis of experiment results to answer the research questions proposed.

3 Literature Review

3.1 What is Ransomware?

Ransomware is a type of malware with a specific goal of extorting payment from a victim by either using some type of scareware, encryption of files or data exfiltration [3].

A. Liska & T. Gallo, (2017) define ransomware as “a blanket term used to describe a class of malware that is used to digitally extort victims into payment of a specific fee“.

There have been various types of ransomware over the years but two main types stand out as being the most prevalent. Crypto ransomware uses cryptographic encryption methods to prevent the victim from opening important files. Locker ransomware locks the victim from their machine [5]. Both types attempt to force the victim to pay a ransom to either unlock their machine or access their files. Once the payment is made, the victim may be given a decryption key or a decryptor application that will decrypt the files or unlock the machine. Not every ransom payment

results in the victim being able to recover their files, only around two-thirds of victims who paid the ransom are able to recover their files [6].

Crypto ransomware uses different types of encryption keys and methods to carry out it malicious actions. Symmetric and asymmetric keys are used in various ways to facilitate the encryption. In some instances, these keys are obtained through the connection to a command-and-control server in which the ransomware communicates over a network or internet connection.

From 2021 ransomware groups switched from targeting individuals to focusing on larger corporations. This switch resulted in higher ransomware profits for the ransomware groups as they were able to reach larger networks of target machines which are critical for organisations to carry out their functions [3]. This worrying trend has spread to organisations like the Nation Health Service in the UK (NHS) when in 2017 the WannaCry ransomware strain caused chaos in around 48 hospitals [5]. Targeting hospitals goes some way to explaining the morals and ethics behind some of the ransomware groups.

The emergence of cryptocurrencies and especially privacy-enhanced cryptocurrencies like Monero exacerbated the ransomware trend and made it easier for ransomware groups to make vast amounts of money from their campaigns. This is further fuelled by the focus on Ransomware as a Service.

Ransomware-as-a-Service allows ransomware groups to provide turn-key solutions to other bad actors that want to extort victims, but don't want to go to the effort of developing their own ransomware solution. This has become an entry point for criminals on the dark web and has lowered the barrier to entry for these malicious actors [4]. The RaaS provider typically take a 20-30% cut of the extortion payments and is a very lucrative, lower risk strategy for ransomware groups [4].

The increasing re-use of code within these ransomware groups provides a good opportunity for defenders to carry out both static and dynamic analysis of the malicious code to improve identification techniques surrounding ransomware strains and gain an insight into the operation of the ransomware groups and their malicious software.

A timeline of ransomware strains has been provided in figure 2 to outline the progression of ransomware strains over three decades.

Year	Event
1989	The first known and documented ransomware: *AIDS Trojan* (also known as the *PC Cyborg* virus)
2006	*GPCode* and Archiveus Trojan began using more powerful asymmetric RSA encryption.
2013	*CryptoLocker*, one of the most damaging ransomware, possibly profited US$27 million
2014	*ScarePackage* ransomware was the first to target Android mobile platforms. It infected users as fake apps appearing to be Adobe Flash or other well-known antivirus, and pretended to scan user files when launched.
2015	*CryptoWall 2.0*, delivered via emails, PDF and various exploit kits, used TOR to obfuscate C&C communications, and incorporated anti-virtual-machine and anti-emulator techniques to avoid analysis via sandboxing.
2015	*Linux.Encoder* was considered the first known ransomware targeting the Linux platform.
2016	*KeRanger* was discovered as the first known ransomware targeting the MacOS platform.
2016	*PoshCoder* became the first fileless ransomware and instead used PowerShell commands to encrypt files.
2017	*WannaCry* attack propagated through the Windows EternalBlue exploits, causing an estimated USD$4billion of damage globally through loss of data and disrupted business processes.
2017	*NotPetya* was the first known ransomware to encrypt the Master File Table of the NTFS drive.
2020	*RagnarLocker* was the first known virtual-machine-based ransomware to be deployed as virtual machines and to encrypt host files via shared folders, in order to evade host-based ransomware detection.
2020	*Maze* was the first known ransomware to exfiltrate sensitive data to blackmail users into paying ransom.

FIGURE 1 -

3.1 Landscape

The landscape of literature related to ransomware has increased dramatically in recent years. This reflects the increased number of attacks carried out, and the sophistication behind each attack. There is a desire to find a mitigation strategy that will stop future ransomware impacting organisations in such a devastating way. One of the motivations for defence is to attempt to stop the encryption phase of the attacks by using various strategies aiming to either block encryption or to allow recovery of encrypted files. The stage of detection becomes important with ransomware and various literature has outlined the importance of early detection.

Ispahany et al., (2024) carried out a survey of papers on ransomware detection and found that real-time and early detection techniques are neglected in the studied literature. The importance of early detection in minimising the damage caused by the ransomware has been highlighted in previous studies [8]. Early detection is therefore an important area of research when it comes to ransomware.

Detection based on Windows API or syscalls are prevalent in the studied literature and are traditionally the preferred method for detection techniques. Operating system API function calls are a mechanism by which developers can use built in operating system functions to carry out actions within their software. Syscalls are the kernel mode system calls that software running in user mode execute through the use of higher-level function calls. Various studies have shown that API function and syscalls can be used as a detection mechanism for ransomware or malware in all operating systems from Windows, Linux and Android for example [9], [10], [11] and [12].

Detection software is able to monitor API function calls in real time to build a pattern of ransomware behaviour that can be used to classify malicious activity. The models of ransomware behaviour can be built up using a variety of methods, one of which is the dynamic analysis of different ransomware variants to gather API function call information during execution. The output of this dynamic analysis can then be fed into machine learning models to be used in ransomware classification [7], [13], [14].

It is worth noting Ispahany et al., (2024) consider detection of API function calls relatively late in the attack lifecycle if they are based on the cryptographic API calls. This is because, by the time the ransomware may have made cryptographic API calls it will be too late to prevent damage to the victim's machine or files. In addition to this ransomware may not use built in OS function calls to carry out its encryption, therefore bypassing any hooks on cryptographic API functions. AV/EDR systems therefore usually employ hooks in lower-level libraries to make sure they catch all API calls before being turned into syscalls. This is explained more in depth later in this work.

As well as the output of dynamic analysis, Windows API function hooks are employed by real time detection systems built into AV/EDR systems [15]. These detection systems employ user land hooks to monitor running processes and insert themselves into user land process. This allows the AV/EDR systems to intercept specific function calls to execute other heuristics which determine whether the running process is carrying out malicious behaviour.

If ransomware is able to disable or avoid AV/EDR hooks, then it will affect both the ability for dynamic analysis to generate useful behavioural data based on Windows API calls and also the ability for real time detection systems to intercept malicious behaviour.

3.1 Stages of a ransomware attack

Several techniques have been proposed to detect ransomware within various stages of the ransomware attack chain. Begovic et al., (2023) carried out a survey in which they identified four

phases of attack: initial compromise, establishing a foothold, encryption and extorsion as shown in figure 1.

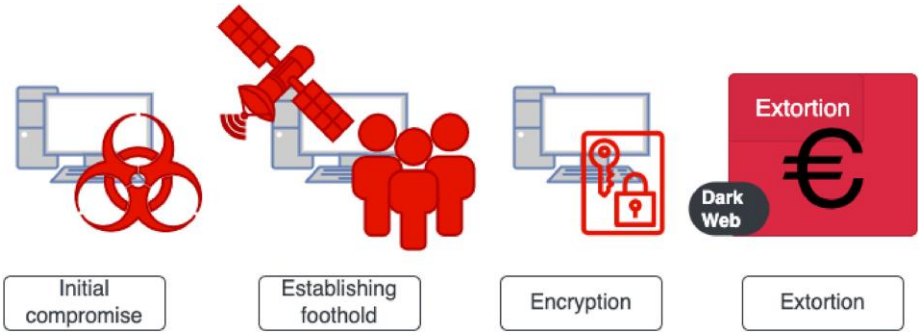


FIGURE 2 - FROM KENAN BEGOVIC, ABDULAZIZ AL-ALI, QUTAIBAH MALLUHI

The Mitre Att&ck framework includes a more granular breakdown of the attack stages [17]. The Att&ck framework gives insight into the tactics and techniques used by malicious actors in each phase of their attack. This work evaluates the Att&ck framework to understand the techniques to focus on in each stage of the attack, this will provide guidance on which API calls would be likely to be employed by ransomware.

Initial compromise of a victim's computer is the first stage at which ransomware can take hold of a host or network. This usually involves techniques and tactics to get the first stage of the malware onto the victim's computer. This can take the form of phishing, spear phishing, malware on web pages and exploited vulnerabilities in operating systems or software [16]. The Att&ck framework outlines various techniques including the supply chain compromise technique, which is becoming more popular in recent times.

Initial compromise is interesting in regard to ransomware because "loader" or "dropper" software is often used as the first stage once the victim's machine has been compromised. Ransomware uses loader software to get its other malicious binaries on the machine without detection. For example, a loader called Pikabot has been used in the BlackBasta ransomware campaign for initial compromise allowing further undetected download of malicious payloads like encryptors and data exfiltration tools downloaded [18], [19]. Pikabot uses function hooking evasion techniques discussed later in this literature review.

Once a machine has been compromised, the ransomware moves onto the next phase of attack which includes establishing a foothold on the victim's machine or the within the victim's network. This stage relates to the execution phase in the Att&ck framework and will be the point when early detection by use of API function calls starts to be feasible. Within the execution category of the Att&ck framework there is the technique of Native API calls [20], this is the technique employed by ransomware to carry out its functionality. Native API calls allow the ransomware to carry out useful functionality in the operating system like encrypting files or exfiltrating data through the network. The ransomware may use documented high level Windows API functions from the Windows API libraries or it may also use native function calls such as the ones starting with Nt* and Zw* in the NTDLL.DLL library to carry out its functions. As all API calls are funnelled through native libraries such as NTDLL.DLL, a lot of AV/EDR systems hook the functions inside the native libraries to monitor the behaviour of processes running on the system.

During the establishing a foothold phase the ransomware may also carry out other techniques defined in the Att&ck framework which tend to be persistence, privilege escalation, defence evasion, credential access, discovery and lateral movement.

By the time the ransomware has reached the encryption, and extortion phases the damage has been done and the victims files will be very hard to recover at this point without negotiation of a decryption key with the ransomware group. Data may have also been exfiltrated from the machine to a storage location owned by the ransomware operators. The aim of any defence against ransomware would be to prevent the ransomware advancing to this stage as there is very little that can be done after data have been encrypted or exfiltrated.

There is a heavy focus in the literature around designing pre-encryption algorithms based on machine learning that focus on malware classification.

Kok et. al., (2019) [14] propose a pre-encryption algorithm based on a machine learning algorithm trained on API function calls. They mention that the API function call monitoring was stopped when a cryptographic API call was encountered, they however fail to mention how they determine a cryptographic API call. The problem with this is twofold; the first issue is how to determine what is a cryptographic API call, as variants of ransomware use different sets of APIs along with third party libraries and custom cryptographic functions. The second is that a key feature of crypto-ransomware is the fact that it uses encryption functions, therefore, to identify crypto-ransomware it might only be possible by monitoring for encryption activity.

These algorithms aim to detect ransomware during real time analysis of processes executing on the victim's machine. The AV/EDR software must have a way to intercept process behaviour before it can begin to utilise the detection algorithms. The techniques to intercept and evade interception is the focus of this work.

3.3 Characteristics of ransomware

The characteristics of ransomware include the encryption of victim's files after the initial compromise and establishing foothold phases have been executed on the target machine. This is not the only form of ransomware, but in modern times this is the most prevalent [16].

Data exfiltration is another characteristic of ransomware and is increasingly becoming a major focus for ransomware groups beyond encryption, to the point that some current ransomware groups solely focus on data exfiltration, avoiding encryption techniques entirely. In Q1 of 2020 5% of ransomware was employing data exfiltration, compared to 90% in Q1 of 2022 [21]

In this work we are interested in exploring the methods used for execution of ransomware payloads while establishing a foothold on a victim's machine, specifically the methods used by ransomware to evade the detection techniques used by AV/EDR systems.

Dargahi et al., (2019) [5] propose a ransomware feature taxonomy based on the Lockheed Martin Cyber Kill chain (CKC).

The Cyber Kill Chain contains the following phases: reconnaissance, weaponization, delivery, exploitation, installation, command and control and actions on objectives. These are shown in figure 3.

Dargahi et al., (2019) [5] provide a ransomware taxonomy based on the Cyber Kill as shown in figure 4.

As you can see from the taxonomy there are several distinctive features that can be used in identifying ransomware based on the different phases of the kill chain.

These features can be categorised as: file access patterns, encryption methods, evasion techniques, backup encryption, command and control and ransom payments.

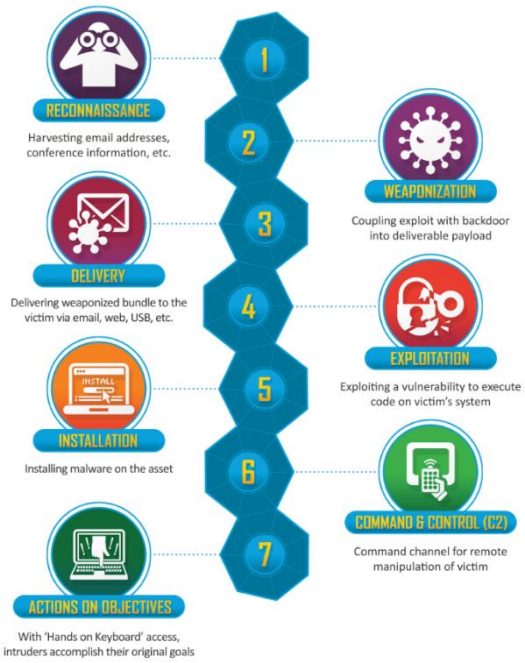


FIGURE 3 - LOCKHEED MARTIN CYBER KILL CHAIN

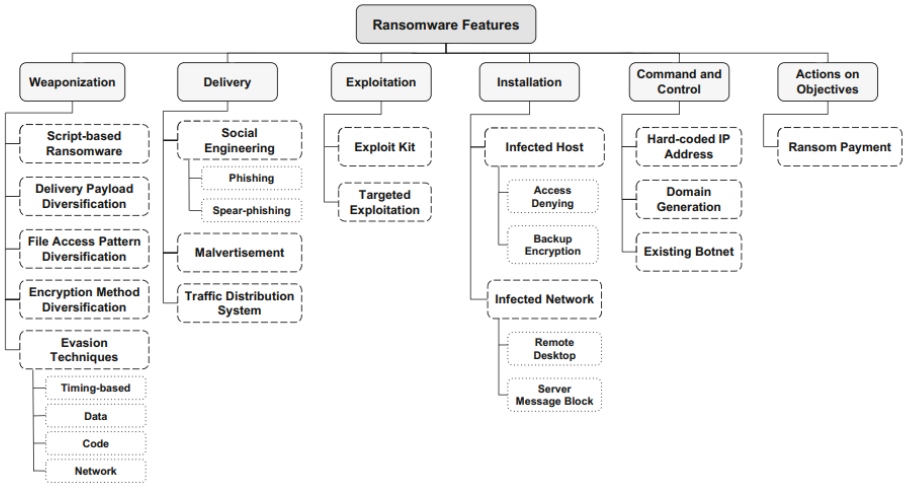


FIGURE 4 - RANSOMWARE TAXONOMY BASED ON THE CYBER KILL CHAIN

Kok et al.,(2019) mention that the encryption is an important characteristic of crypto-ransomware for obvious reasons, however other studies have shown that attempting to detect ransomware using encryption monitoring incurs a loss of files before detection can happen. For example, [22] found that their detection method based on detecting encryption from file entropy resulted in some loss of user files in certain cases. Detection in the pre-encryption phase is more desirable as this should prevent any data loss, however this proves difficult as it is hard to distinguish between benign and malicious workloads, especially using the techniques outlined in this work regarding analysis of Windows API calls.

This work focuses on the code section of evasion techniques defined in the taxonomy from figure 4. This is where ransomware is able to bypass AV/EDR detection techniques to carry out its

functionality. This stage is crucial for detection of ransomware before it moves onto more destructive stages such as encryption and data exfiltration.

The classification between malicious and benign processes is determined by techniques such as the training of machine learning classifiers. However, it is important that AV/EDR systems can intercept ransomware behaviour so it can execute these machine learning algorithms in the first place. The AV/EDR system needs to insert itself between the code executing Windows API calls and the operating system. Without an entry point to intercept malicious processes any detection system is rendered blind to the ransomware behaviour.

3.3 Tactics and Techniques employed by current ransomware

In their paper Lewis T, Rimal B et al. (2024) [2] identify the Mitre Att&ck techniques that could potentially be evaded by removal of user-land hooks used by AV/EDR tools.

To detect current ransomware based on Windows API function calls it is useful understand the techniques that ransomware variants use, to provide guidance on what sort of Windows API functions may be used for each technique. For example, knowing that ransomware would likely employ data exfiltration means that AV/EDR systems could focus on detecting network-based calls such as Windows socket functions. This section outlines the different techniques from the Mitre Att&ck framework to provide context around the techniques used by ransomware.

AV/EDR systems are at a disadvantage for identifying all attack techniques possible from malware using function hooks. This is because hooking every function within the Windows API will significantly slow the performance of the system. Lewis & Rimal, (2024) [2] found that not all exported Windows API functions are hooked by many commercial AV/EDR vendors. They also note that not all API functions are relevant to identify attack techniques, so therefore not worth the performance hit of hooking. Many vendors take the approach of only hooking API calls that provide the highest fidelity for attack technique identification [2].

The literature has many detection mechanisms based on API monitoring [10],[11],[9],[23],[12]. All the current approaches create datasets from dynamic analysis to be used to train machine learning models, which in turn allows classification of malicious and benign executables. This method has been shown to be effective at detection in many studies. This work however does not focus on detection via dynamic analysis methods, instead it focuses on Windows API hooking to provide real time interception of function calls into the operating system. Classification of malicious behaviour is the next step after interception has occurred.

We acknowledge that static analysis will also play a part in the identification of function calls and Windows libraires, but this work avoids any static analysis techniques as static analysis has limited usefulness when ransomware can use obfuscation on the binaries.

The following sections break down the tactics from the Mitre Att&ck framework [20] used by ransomware to identify the potential threats posed by user-land function hook evasion.

Execution

Execution of ransomware comes in many forms, from user execution by way of opening an .exe file to scheduled tasks and scripts. Some ransomware strains carry out tasks using a .BAT file executed by command.exe which carry out various functions before executing the main process. A full list of execution techniques is provided at [24].

Once the main executable is running, execution of the ransomware functions can be carried out using the Native operating system API functions. For Windows based ransomware this means that user mode API calls made through the Windows API interface. These function calls can be

monitored and identified. Specific API calls may be used to create threads and processes from the main process [20].

There are various mechanisms by which AV/EDR systems can detect the execution of a process. For example, one technique is for AV/EDR system to inject code into the explorer.exe process using the CreateRemoteThread function call and then hook the CreateProcessInternal which is called whenever any process is started [15].

Persistence

Persistence techniques aim to allow the victims machine to remain infected with the ransomware, even after the machine has rebooted. A full list of persistence techniques can be found at [25]. A common technique is to automatically execute the ransomware binary via boot or logon autostart execution [26] This is commonly done via modifications to the windows registry. Another technique is to create Windows services that point to the ransomware executable. API calls to modify registry entries can be monitored to aid in ransomware detection.

Defence Evasion

Defence and evasion techniques allow the ransomware to avoid being detected by any anti-virus or EDR system in place on the victim's machine. Evasion techniques are vast and vary wildly between ransomware variants but there are some common tactics outlined in the Mitre Att&ck framework [27]

The ransomware may make modifications to the Windows registry to hide configuration information or hiding of keys by prepending a null character. The ransomware may also evade detection when it places files or security objects on the machine by modifying the file attributes to either hide the files within the system or to hide additional malicious user accounts. Ransomware often make uses of virtualization or sandbox environment detection which allows it to avoid being analysed by researchers.

The focus of this work is the evasion tactics related to the evasion of the user-land hooking techniques employed by AV/EDR tools. The ransomware may use un-hooking of AV/EDR hooks, direct and indirect syscalls to bypass the hooks combined with other techniques such as process injection. These techniques are further later in this work.

Lateral movement

Once ransomware has established a foothold on the victim's machine one possible path the ransomware can take is to attempt movement around the network trying to infect other machines. This can be achieved via command and control from the ransomware operators or the ransomware executable itself attempting to self-propagate to adjacent network devices.

A spike in network-based API or system calls would be a good indicator for detection of lateral movement to other devices. The API call sequence can be used to detect lateral movement techniques, for example Elgohary N et al. (2022) [28] detect lateral movement of the Mimikatz tool from the API call sequence. This detection of lateral movement would need AV/EDR tools to hook the API calls mentioned in [28] or at least the native NTDLL.DLL functions they resolve to.

Command and control

Command and control mechanisms are employed by ransomware to allow remote control of the ransomware executing on a victim's machine. The use of network and in particular TCP or UDP protocols to communicate with the command-and-control server would mean that the API calls would be similar to lateral movement and utilise the network API calls. Command and control servers will most likely have a hostname on the internet therefore any ransomware may make use of DNS API calls.

Impact

In crypto-ransomware, encryption of the victim's files is the main impact. These encryption methods vary between ransomware variants. The methods range from using built in Windows API calls to custom encryption code embedded in the malware. Custom encryption code will not show up in any API call or system call monitoring but will instead show large amount of file modification calls. The high number of file modifications is a good indicator of ransomware and can be used for detection.

Data Exfiltration

Data exfiltration is becoming more popular with current ransomware, and some are even focusing more on data exfiltration than encryption. Ransomware operators see data exfiltration as a more profitable means of extortion and in some cases abandoning encryption altogether [29].

There are a number of legitimate tools that can be used for data exfiltration and are included in ransomware attacks. A list of the most common tools is detailed by [21]. These tools include: rclone, AnyDesk, RDP, CobaltStrike, ScreenConnect, Atera, WinRAR and more. Figure 5 from [21] outlines how these tools are distributed across ransomware variants.

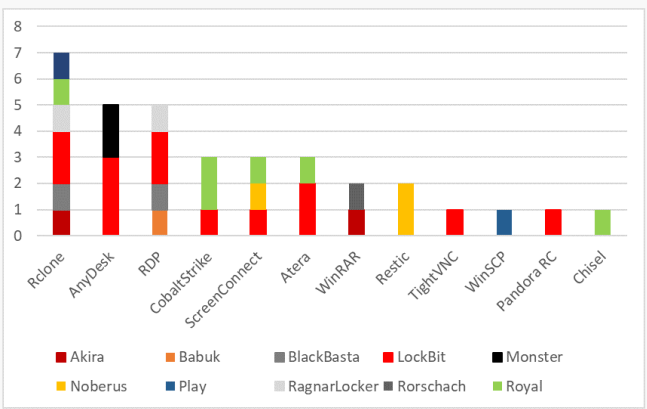


FIGURE 5 - POPULAR TOOLS USED FOR DATA EXFILTRATION

As these tools can be used in legitimate circumstances it is important to identify how the tools are used for used in decision mechanisms. If a detection system is going to hook on specific function calls, it needs to have as much information as possible to make a decision on whether to stop the data transfer.

These decisions can be made using a list of the most popular data exfiltration sites used by ransomware. This is complicated by the fact that attackers tend to favour the use of trusted domains for their exfiltration in a technique called "Living off trusted sites (LOTS)"[30]. However, with proper planning the most relevant sites can be screened for and can be useful in identifying data exfiltration activity.

From [31] useful Windows API calls to hook would be around networking, specifically windows socket functions. For example, the functions WSAStartup(), socket(), bind(), listen(), accept(), shutdown(), connect(), send() from mswsock.dll, ws2_32.dll and wsock32.dll libraries.

Encryption

Current studies based on detection of encryption seem to lack a comprehensive scope of encryption methods which could be employed by the ransomware executable. These methods include using built in Windows API or syscalls to call built in operating system functions for

encryption, third party library encryption and finally custom encryption code which doesn't use any OS API calls to do the file encryption.

Encryption specific calls relating to the Windows WIN32 API are any calls that come from the wincrypt.h and winbase.h header files, the most relevant calls would be CryptAcquireContextA, CryptAcquireContextW, CryptCreateHash, CryptEncrypt, CryptGenKey, CryptGenRandom, CryptReleaseContext, Encrypt ilea and DecryptFileA.

Dargahi et al., (2019) found the majority of ransomware samples used standard encryption modules built into the operating system. They mention Cerber v6 and Petya use the built in Windows API calls, for example the CryptGenRandom function from the Windows API.

Not all ransomware will use standard built in APIs however, they may use custom built encryption code which will not be so easy to detect using the technique of function hooking.

Some ransomware will also use a higher level framework which may or may not call standard API functions. An example of this is the fact that on Microsoft Windows a lot of modern ransomware is built using the Microsoft .NET Framework (Cicala & Bertino, 2022).

Cicala & Bertino, (2022) carried out an analysis of encryption key generation for modern crypto-ransomware and found that for example the CryptLocker ransomware which has a number of clones such as CryptoWall, Crypt0L0cker and TorrentLocker, uses the AesCryptoServiceProvider class in the Microsoft .NET framework which in turn calls the CryptGenRandom Win32 API function.

Cicala & Bertino, (2022) also analysed the Petrwrap ransomware and found that it used the RNGCryptoServiceProvider class from the Microsoft .NET framework. This raises an issue when looking for encryption functions to hook in that some ransomware use framework to develop their software. For instance, the CryptLocker and Petwrap samples discussed above both use the Microsoft .NET framework. It is difficult to know which functions the cryptographic libraries within these frameworks use. To get around this, functionality is built into the simulation software developed for this work to make simulated calls to each cryptographic library within the framework, while monitoring software analyses the Win32 API calls that are called from the framework. The result of this is outlined in the experiment section of this work.

3.2 Detection techniques used by AV/EDR Systems

It is possible to detect ransomware at various stages of the cyber kill chain defined earlier. The goal of this section is to define the various points of detection for AV/EDR tools and gain an understanding of where API call monitoring fits into the attack chain.

Moussaileb et al. (2022) [33] break down the attack into the four stages which are analogous to the stages defined in the Cyber Kill Chain [16]. The stages defined are delivery, deployment, destruction and dealing. When it comes to ransomware detection using API function call monitoring, we can see this detection technique falls into the deployment phase of the taxonomy defined in figure 6 [33].

Figure 6 shows various techniques that can be used in detection for the delivery and deployment phase of an attack. This work considers the phase of destruction to be the point of no return and detection techniques must deal with any pre-destruction phase which we define as pre-encryption.

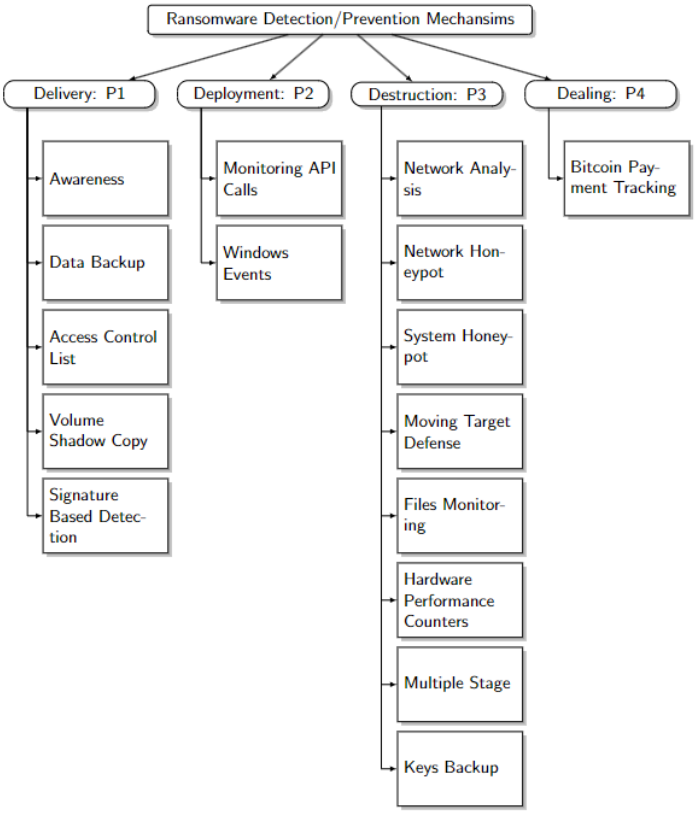


FIGURE 6 - MOUSSAILEB ET AL. 2022

The techniques in the delivery phase really come down to prevention methods implemented by the system as a whole. Awareness is about training users and system administrators to spot the signs of an attack, for example training on looking out for phishing emails and not opening links from emails unless the sender is verified. Backup and access control lists are prevention methods implemented by the system administrators to prevent unauthorised access and to allow recovery should other techniques fail. The only real detection techniques that falls into the delivery phase is signature-based detection.

Signature based detection can be used at the firewall or edge device to prevent malicious software arriving at the endpoint. It can also be used on the endpoint itself in the case that network security is circumvented.

Signature based detection however has been shown to be easily defeated by techniques such as obfuscation in the form of packing and other techniques. In addition to this signature-based detection needs to be constantly updated with new signatures as new variants of ransomware are created.

Considering this, most detection systems in the modern era use a mixture of detection techniques based on dynamic behavioural based detection. In behavioural based detection the detection system monitors resources accessed by any process running on the system. It uses pattern recognition based on dynamic analysis carried out on ransomware samples in a sandbox environment. These patterns are built into the detection system to identify ransomware when it is executed.

The deployment phase in figure 6 identifies the technique of monitoring API calls. This is the detection technique that this work will be focused on and is discussed further in the following sections.

3.2 Detection timing

Mehnaz et al., (2018) [34] mentions that one of the limitations of ransomware detection is the timing in which detection is carried out. Detection and subsequent prevention of file encryption is essential to make sure files are not lost. This means that ransomware protection software would ideally block file encryption at the system level and subsequently block any deletion of files. It is not the case that protection systems can just block encryption function calls, as first the protection system needs to establish whether the process is attempting malicious behaviour.

Most of the literature focuses on training machine learning models which can be called from the hooked API function to determine malicious behaviour. Dynamic analysis of Windows API calls ideally focuses on being able to detect malicious behaviour early in the ransomware lifecycle. Some models focus on dynamic analysis of Windows API calls before encryption functions are called. For example, Kok S et al (2019) [14] proposed a pre-encryption detection algorithm based on API and system calls before the first encryption function is called, therefore training their models to be based on dynamic analysis before encryption happens.

Their work emphasizes the importance of preventing encryption to minimize loss of data. Their work however doesn't mention what encryption functions they detect and can be easily bypassed by ransomware that uses custom encryption methods or non-OS based encryption functions such as third-party libraries. This highlights the fact that algorithms based on machine learning models for detection can be skewed by any evasion techniques employed by the ransomware process.

This work focuses on antivirus software detecting ransomware using function hooking of Windows API function calls and the subsequent evasion techniques by the ransomware; however, based on the fact that evasion techniques can skew dynamic analysis models, a real antivirus detection system would incorporate other techniques to take a more holistic approach. Function hooking is useful as it allows the antivirus software to intercept specific Windows API function calls and therefore can prevent data loss, but this relies on generation of good detection algorithms so that the code inside the function hook has the relevant information to prevent the call from completing in the first place. The algorithms for which the AV software makes a decision is beyond the scope of this work.

3.2 What is function hooking?

As outlined in [15] dynamic heuristic engines employed by current AV/EDR systems use user land or kernel land hooks to detect real time behaviour of running processes.

A function hook is essentially a redirect for a specific function call in the Windows Operating system and is implemented by inserting an assembly JMP instruction into the memory location of a specific function call from the Windows API. This means that when a process uses an API function in its code, the JMP instruction will point them to another memory location where a hooked function is located, instead of the original function location. This results in a redirection of the function call to a memory location located in a hooked library controlled by the AV/EDR software.

Lewis T et all (2024) outline the following steps that AV/EDR systems use to implement hooking [35]:

1. AV/EDR uses one of a few possible methods to inject itself into the startup of a running process.
2. AV/EDR maps it user-land hooking DLL into the process memory space
3. AV/EDR injects the JMP assembly instruction into each API function it wishes to hook that point to the AV/EDR DLL memory location for the hooked function.

4. The user process runs as normal which the AV/EDR system can intercept all hooked function calls.

In 2005 with the Windows Vista release, Microsoft introduced a feature called Kernel Patch Protection or PatchGuard. KPP aims to support the stability of the Windows Operating system by stopping modification to kernel components. Before KPP was introduced AV/EDR vendors could utilise kernel modification to hook kernel level functions and therefore monitor all process operations at the kernel level. Since KPP was introduced AV/EDR vendors have had to rely on hooks outside of the kernel in user land [36].

3.2 Endpoint Detection Function hooking

The practice of hooking the Windows API is heavily used by anti-malware applications such as AV/EDR tools and forms part of a layered defence when it comes to malicious behaviour detection.

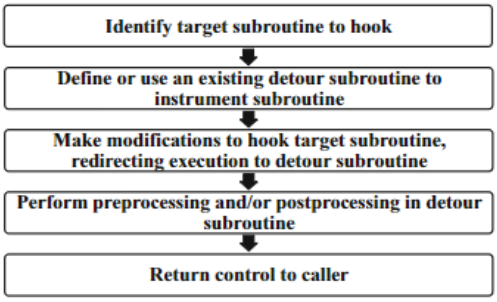
AV/EDR systems have several components that make up the entire solution. The common components that make up these systems are outlined in [15], [1] and consist of:

- Static scanners that perform static analysis of files including the portable executable (PE file). Static scanners can look at the imported libraries and the functions used to assess any possible malicious intent.
- The DLL containing the function hooks which have the job of intercepting the function calls and either monitoring behaviour or executing the ML classifiers that have previous been trained on API calls during earlier dynamic analysis.
- Kernel drivers that are responsible for injecting the hooking DLLs and can in some instances be responsible for hooking kernel level APIs.
- Agent service that aggregates the output of the other components and make decisions based on the real time data. This will also have the responsibility of synchronising data with a centralised management server.

Current antivirus and endpoint detection and response platforms identify malicious behaviour in every running process by injecting user-land hooks in front of Windows API function calls [35]. The technique of function hooking is distinct from other detection capabilities employed by AV/EDR systems. Other techniques include static file signatures and heuristic analysis. These other techniques are not described in this work, but it is worth noting that the detection methods discussed in this work would not be used in isolation.

Hooking is defined by Babun et al., (2017) as the interception of specific functions or system calls to monitor and/or alter the execution of the specified call. Hooking function calls is a good way for detection software to ensure it is notified of any attempt to call that function by another process. The distinction between API and syscalls is discussed below in its own section.

Babun et al., (2017) outline the steps taken by detection software when hooking function calls, this is shown in figure 3.



The first step is to identify a function to hook. Detection software relating to ransomware would be interested in any function that is related to the techniques identified previously in this work such as file modification, encryption, lateral movement and data exfiltration. This is because in theory any process that wishes to carry out malicious behaviour would call the operating specific functions that relate to that behaviour. For example, if a piece of software wanted to encrypt a file it might call the CryptEncrypt function from the Windows API, which would in turn instruct the operating system to carry out the actual encryption of data on the file system. This may not be the case for all ransomware as there are other methods for carrying out malicious behaviour that may circumvent the Windows API functions, this is discussed in the evasion section later.

Other API function calls that may be of interest to detection systems are file system access functions, data exfiltration or network communication. As discussed in the ransomware techniques section earlier, ransomware could make lots of modifications to files for encryption, data exfiltration or other techniques and AV/EDR systems make use of this fact by intercepting function calls used to carry out these techniques. A lot of ransomware variants make timestamp and naming changes to evade or encrypt the victim's files, so Windows API functions that relate to file modification would be an ideal candidate for functions to hook as is the same for network sockets, encryption and registry access.

There are various levels of Windows API calls that a piece of code can use. These start at the documented API level where code can include the C++ header files that allow them to use higher level functions for its capabilities. This is the easiest way to develop techniques within malicious code as the API calls are well known and documented. Code from higher level functions get forwarded onto lower-level native APIs that then make a syscall to the operating system to carry out it functions. The Windows internal architecture is shown in figure 5. As you can see there is a clear boundary between user mode and kernel mode, separated by NTDLL.DLL.

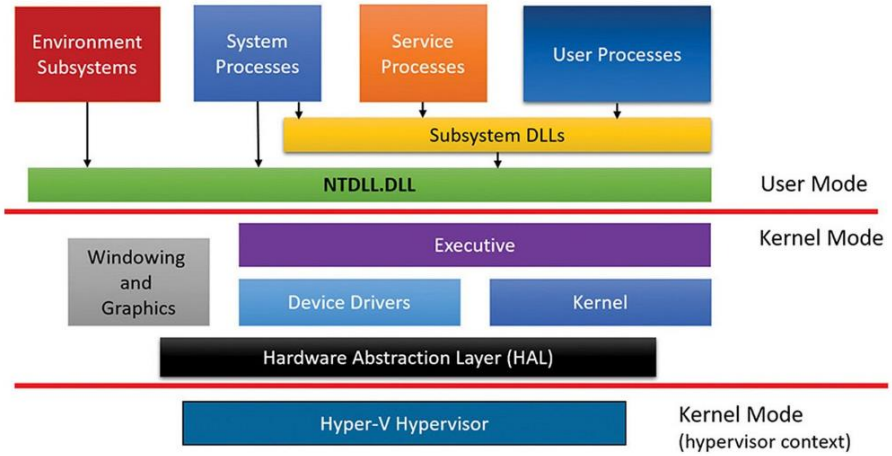


FIGURE 7 - WINDOWS INTERNAL ARCHITECTURE

Malicious code can also call the undocumented native APIs within the Windows Operating system, effectively bypassing any hooks created for the higher level functions. For this reason, AV/EDR systems tend to hook the lower-level native APIs only to make sure they can

catch a wider array of behaviour with less function hooks. Hooking all functions would seriously impact performance of the system. [1]. This work identified which NTDLL.DLL functions current AV/EDR solutions hook.

Antivirus programs use a mixture of userland and kernel function hooks to intercept specific function calls [15]. The hooks are initiated by injecting a special library containing the hooking function into all processes that get executed on a machine. This is typically done in one of three ways:

By using the method of updating the AppInit_Dll registry key to add the DLL that contains the hooking code. This will then inject the hooking DLL into all processes started on the machine. As mentioned in [15], this method is mostly deprecated now.

By injecting the hooking DLL into the explorer.exe process to hook the CreateProcessInternal function. This also allows hooking code to be initiated for all processes but is not always reliable.

Finally, by using a Windows driver that runs in kernel land to register a callback using the PsSetCreateProcessNotifyRoutineEx function [15].

Hooking system calls related to file access and modification, registry access and modification, API un-hooking and encryption would allow detection software to intercept any attempts of ransomware to carry out malicious activity on the victim's machine if a preceding call pattern is met. Once ransomware is detected, hooking the above calls would allow the detection software to prevent the malicious process carrying out it functions by intercepting the function call. This allows the detection software to stop the attack.

The AV/EDR system can also use these hooks to monitor system activity to feed into previously trained ML classifiers which allow more sophisticated detection based on the Windows API call patterns.

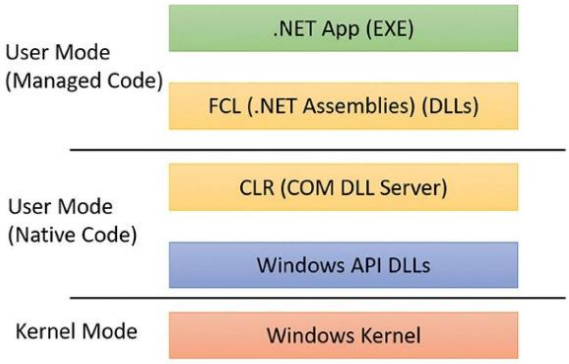
As can be seen from above AV/EDR, rely heavily on hooking user-land Windows API functions, whether at the higher documented API level or at the lower level with undocumented Nt* & Zw* functions from NTDLL.DLL and KERNELBASE.DLL.

3.4 Windows API function calls vs syscalls

The Windows operating system aims to protect critical operating system functions from being modified by user mode applications to maintain operating system integrity and stable functioning. It does this by separating user mode applications from kernel mode applications. All user applications including ransomware executables reside in user land and can only make calls across the boundary to kernel mode by calling functions that exist in "bridge" libraries such as NTDLL.DLL. As can be seen in figure 6 NTDLL.DLL is responsible for making syscalls into kernel mode [1].

Sometimes in the literature, syscalls and API function calls can be used interchangeably such as in [38]. System calls and API function calls are distinguished here as is necessary for this work.

As outlined in [39] Windows API functions are not the same as native system calls in the Microsoft Windows Operating System. Windows API functions are documented, callable subroutines in the Windows API. These function calls execute in user mode and the OS is responsible for translating these function calls to kernel mode system calls.



Syscalls are the bridge between documented user land Windows API calls and kernel land code. Every Windows API call will in turn call a syscall function that passes the call onto the Windows kernel mode drivers for execution.

When a user land application such as a ransomware executable makes a Windows API function call, the function call gets forwarded on to so called “native” functions from specific DLL’s within Windows. These “native” functions in turn make a call into code at the kernel level. These “native” functions sit inside specific libraries on the operating system, namely NTDLL.dll, NTGDI.dll and KERNELBASE.dll. For example, the function OpenProcess in Kernel32.dll calls the function OpenProcess in Kernelbase.dll which in turn calls the “native” function NtOpenProcess in ntdll.dll. The last call to NtOpenProcess initiates the Syscall stub which makes the bridge into the kernel land code.

Function hooking only allows us to intercept the function calls before the actual syscall itself. Due to this fact malware may use the undocumented system calls directly to bypass any AV/EDR function hooks designed to detect its behaviour [40]. This does not mean however that these function calls are executed in kernel mode (Ring 0), as the undocumented system calls are sent to the kernel mode Windows drivers that translate these calls into the underlying operating system syscalls.

If ransomware uses the native DLL’s directly, for example NTDLL.dll, detection becomes harder. This is because the API call output becomes less descriptive and granular than if the ransomware used the higher-level document API calls. Worse than this, the ransomware could bypass NTDLL.dll altogether and make the syscall directly by knowing which syscall number to execute. Therefore, the output from the analysis may include lots of calls to Nt* functions or may not include some function calls at all by by-passing the native function calls altogether and making direct or indirect syscalls into kernel land.



FIGURE 8 - WINDOWS SYSTEM CALL DISPATCHING

Another thing to note is that certain frameworks on Windows may also wrap the Windows API function calls such as in the Microsoft .NET Framework which uses a managed runtime wrapper around the Windows API calls. It may be useful to know if the ransomware uses a framework for development as framework function calls can be mapped to Windows API function calls and used

in analysis. The relationship between user mode, managed and native code vs kernel mode is shown in figure 6.

In their paper on reproducible ransomware analysis [41] a tool called ProcMon is used in their analysis to output function calls. This output will be different from [9] for example who use a Cuckoo sandbox environment to output the Windows API function calls of each ransomware experiment. The output of analysis depends on the tool doing the monitoring. Some tools may only monitor syscalls and others may monitor API calls alongside syscalls. This difference can dramatically change the output of any dynamic analysis depending on whether the ransomware using evasion techniques. In theory the lower level the analysis is done at, for example, documented function calls vs undocumented function calls vs kernel level syscalls the more likely the analysis output would represent the actual behaviour of the ransomware. For this reason, AV/EDR systems tend to use only the native Windows libraries that capture the most useful information.

Types of function hooking

The AV/EDR system can employ two types of function hooking. It can use user-land hooks from documented and undocumented Windows APIs or it can use kernel level hooks which require elevated permissions to execute. AV/EDR solutions currently use minifilter device drivers [42] instead of hooking at the kernel level. This is the result of Microsoft introducing a feature called Kernel Patch Protection [36] which prevents kernel patches in 64 bit versions of Windows [2]. To this end, AV/EDR vendors tend to use a mixture of user land hooking to identify process behaviour and device drivers for detection of malicious activity at the kernel level.

Userland hooks

User land hooks are commonly used by AV/EDR systems to identify real time process behaviour for executing ransomware. User land hooks can be implemented by using the documented Windows APIs, which include functions such as CreateFile or CreateProcess, or the undocumented APIs including all functions that start with the Nt* and Zw* names. The documented APIs are easier to work with as they have documented function signatures that can be easily referenced. The documented APIs are usually accompanied by C++ header files from the Windows SDK which can be used to import the function headers directly into custom code.

Undocumented APIs are harder to work with but offer more capability. Many documented APIs call undocumented APIs to carry out their functions at lower levels. All function hooks have a cost overhead when implanting the function hooking at runtime. The overhead comes from the fact that the system must hook all the identified functions each time a process is started, which impacts the system performance. AV/EDR vendors tend to minimise the function hooks utilized by using lower-level undocumented APIs from NTDLL.DLL. This maximizes their coverage with the least hooked functions.

When the operating system loads a process, it uses a structure embedded into the portable executable format called in the import address table (IAT). When a process is loaded the operating system will fill the IAT with the function addresses of any imported functions from its dependencies. For example, all processes will load the NTDLL.DLL library and its utilized functions will get be mapped into the IAT [1].

There are two popular approaches for hooking user land functions, the first is the use of a JMP or CALL assembly instructions and the second is the manipulation of the IAT [1].

JMP and CALL instructions are generally used to hook the native NTDLL.DLL function calls. In [1] it is observed that when an AV/EDR tool is present on the machine it can be seen that a JMP instruction is inserted as the first instruction to the function NtCreateProcess. This results in redirecting any call to NtCreateProcess to the address supplied by the JMP instruction. The result of this is that the hooking DLL with the redirected address will get a chance to handle the function call before forwarding the call onto the real function. This process is shown in figure 7.

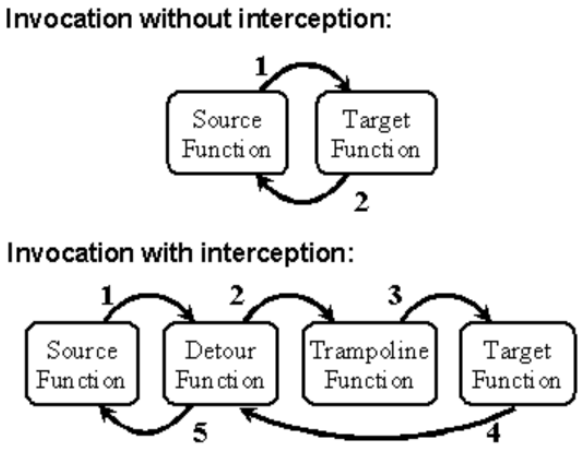


FIGURE 9 - FUNCTION HOOKING FROM MICROSOFT DETOURS

Microsoft provides the C++ detours library which implements the JMP instruction for any targeted API functions [43]. The detours library uses the concept of a trampoline function that preserves the instructions for the target function. The trampoline function allows control to be returned to the target function without interception and also allows pre and post-processing by the hooked function. This is illustrated in figure 8

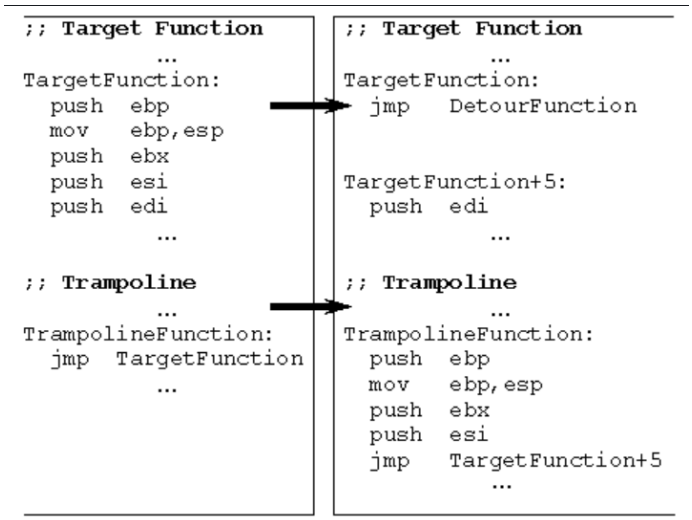


FIGURE 10 - JMP INSTRUCTION INSERTED BY THE MICROSOFT DETOURS LIBRARY

Hooking example

This example demonstrates function hooking using the Microsoft Detours library. Figure 9 shows C++ code that implements a DLL to hook the Sleep() function from the Windows API.

```c
#include <stdio.h>
#include <windows.h>
#include "detours.h"

static VOID (WINAPI * TrueSleep)(DWORD dwMilliseconds) = Sleep;

VOID WINAPI TimedSleep(DWORD dwMilliseconds)
{
    printf("Oh no your not! Sleeping for %lu milliseconds\n", dwMilliseconds + 1000);
    TrueSleep(dwMilliseconds);
}

BOOL WINAPI DllMain(HINSTANCE hinst, DWORD dwReason, LPVOID reserved)
{
    if (DetourIsHelperProcess()) {
        return TRUE;
    }

    if (dwReason == DLL_PROCESS_ATTACH) {
        DetourRestoreAfterWith();

        DetourTransactionBegin();
        DetourUpdateThread(GetCurrentThread());
        DetourAttach(&(PVOID&)TrueSleep, TimedSleep);
        DetourTransactionCommit();
    } else if (dwReason == DLL_PROCESS_DETACH) {
        DetourTransactionBegin();
        DetourUpdateThread(GetCurrentThread());
        DetourDetach(&(PVOID&)TrueSleep, TimedSleep);
        DetourTransactionCommit();
    }
    return TRUE;
}
```

FIGURE 11 - DETOUR DLL USING MICROSOFT DETOURS

Figure 10 shows the code to implement a simulation executable that calls the Sleep function from the Windows API.

```c
#include <windows.h>
#include <stdio.h>

int main(int argc, char* argv[])
{
    printf("Sleeping for 5 seconds\n");
    Sleep(5000);
    return 0;
}
```

FIGURE 12 - SIMULATOR THAT CALLS THE SLEEP() FUNCTION

Figure 11 shows the disassembled assembly while the simulator is executing. You can see that the sleep function proceeds as normal without function hooking.

```
KERNELBASE.dll:76555E00 ; ---------------------------------------------------------------------------
KERNELBASE.dll:76555E00
KERNELBASE.dll:76555E00 kernelbase_Sleep:
KERNELBASE.dll:76555E00 mov     edi, edi                        ; CODE XREF: KERNEL32.DLL:kernel32_Sleep↑j
KERNELBASE.dll:76555E00                                         ; DATA XREF: KERNEL32.DLL:off_75070E0C↑o
KERNELBASE.dll:76555E02 push    ebp
KERNELBASE.dll:76555E03 mov     ebp, esp
KERNELBASE.dll:76555E05 push    0
KERNELBASE.dll:76555E07 push    dword ptr [ebp+8]
KERNELBASE.dll:76555E0A call    near ptr kernelbase_SleepEx
KERNELBASE.dll:76555E0F pop     ebp
KERNELBASE.dll:76555E10 retn    4
KERNELBASE.dll:76555E10 ; ---------------------------------------------------------------------------
```

FIGURE 13 - ASSEMBLY CODE OF SIMULATOR WITHOUT HOOKS

The DLL containing the hooked function definition is now injected into the simulator process using a tool from the Detours library.

Figure 12 shows the disassembled instructions after the hooking DLL has been injected into the process.

Instead of the Sleep() function proceeding normally there is a JMP instruction pointing to our TimedSleep function from the hooking DLL.

```
KERNELBASE.dll:76555DFF db 0CCh
KERNELBASE.dll:76555E00 ; ---------------------------------------------------------------------------
KERNELBASE.dll:76555E00
KERNELBASE.dll:76555E00 kernelbase_Sleep:
KERNELBASE.dll:76555E00 jmp     hooker_TimedSleep               ; CODE XREF: KERNEL32.DLL:kernel32_Sleep↑j
KERNELBASE.dll:76555E00                                         ; DATA XREF: KERNEL32.DLL:off_75070E0C↑o
KERNELBASE.dll:76555E00 ; ---------------------------------------------------------------------------
```

FIGURE 14 - ASSEMBLY CODE OF SIMULATOR WHEN HOOKS ARE INSTALLED

When the Sleep() function is called from the simulator after the hooks are injected, the function call will get directed to the TimedSleep function shown in figure 9. Within the TimeSleep function any arbitrary code can be executed. This is how AV/EDR tools intercept the behaviour of function calls within their detection systems.

Kernel level hooks

Kernel level hooks are generally used by AV/EDR vendors for the purpose of injecting the user land hook DLL into the process memory at startup time. For example, the AV/EDR tool may install a device driver to register a callback to the PsSetCreateProcessNotifyRoutineEx function which gets called each time a process gets started by the operating system. This callback would handle injecting the user land hook DLL into the process memory to register the function hooks for that process [15].

3.8 Detection based on function hooking

It has been described why an AV/EDR tool would use function hooking, and how the function hooking works. This section briefly describes how detection based on function hooking may work. As most AV/EDR are propriety black boxes we do not know the exact heuristics each tool uses to detect malicious behaviour, below is an idea of how a simple mechanism may work.

The goal of a detection mechanism based on function hooking is to hook specific API calls to allow the detection software to monitor the behaviour of processes running on the host machine. This should allow the detection software to use its algorithm to detect ransomware based on the function call patterns outlined in this work.

Function hooking is used by dynamic analysis environments for use in tracing the behaviour of processes when they are executing. The goal is to extract as much information from function calls as possible to feed algorithms that are trained to identify patterns of malicious behaviour in other processes. In this scenario, it is likely that a vast number of functions hooked will be employed

and logged during the dynamic analysis. This is the technique employed by a number of studies that aim to create machine learning models to predict ransomware behaviour [10] [11][44][9][23][12]. Current malware analysis sandbox environments such as Cuckoo and AnyRun also utilize function hooking for this reason.

Dynamic system call analysis from the specific hooked functions will be fed into the detection algorithm and will use information based on the API function calls to decide whether the process is likely to be ransomware attempting to use its techniques for encryption, evasion, or any other feature identified.

A simple example of this process is outlined in figure 15 where a simple function call pattern is used as a signature for malicious behaviour. Real AV/EDR systems would use a more sophisticated approach to detection, but this illustration serves to distinguish between ustilizing function hooking for real time interception of API calls and the use of function hooking as output of dynamic analysis.

In the diagram when the ransomware uses the function LoadLibraryA the detection software intercepts the call. The detection software can then run its algorithm for determining whether the behaviour is malicious. In this example the detection software is using a simple signature of loaded libraries to determine malicious behaviour. This is based on the fact that if a series of libraries are loaded in this order, we can determine that the processes is attempting unusual behaviour that matches other ransomware behaviour.

In reality the detection software would use a more sophisticated algorithm, usually based on a machine learning classifier that has possibly been training on the output of API function calls.

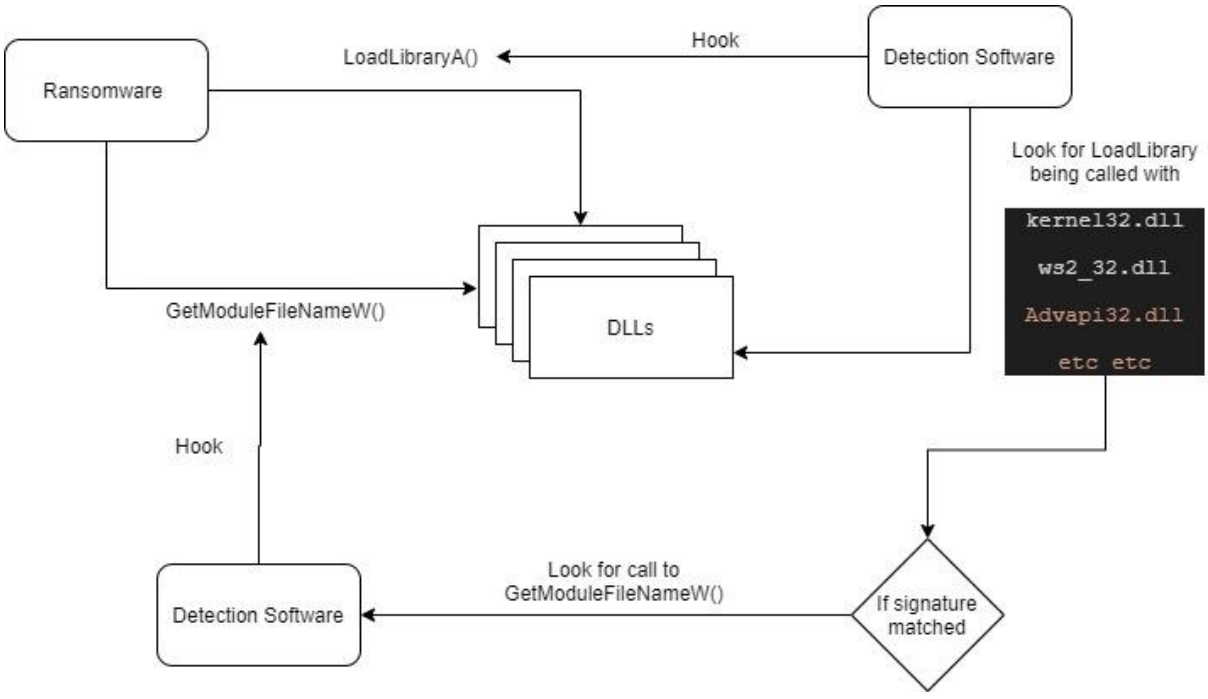


FIGURE 15 - DETECTION BASED ON FUNCTION HOOKING

From this illustration we can see that if the ransomware was able to call the LoadLibraryA function without being detected then the detection software would have no way to execute its algorithms to determine malicious behaviour. We know that current ransomware utilizes evasion techniques which will be discussed in the next section.

In regard to the classification stage Mohan Anand et al., (2022) have carried out an analysis of function calls for 46 different families of ransomware. They identified important API calls for ransomware detection including analysis of the behaviour from the function calls. Their work aims to provide a dataset that can be used by incident responders when monitoring function calls for detection. This type of dataset can also be used to create machine learning models and build detection systems. As far as can be known from the literature the classification was done over all the API calls throughout the ransomware lifecycle, therefore including encryption API calls. The limitation here is that and detection mechanisms created from this analysis will not necessarily be at the pre-encryption phase.

3.6 Evasion techniques

In this section we will discuss the techniques that ransomware uses in an attempt to evade detection by AV/EDR tools. Earlier it was shown that API function call monitoring is a detection technique that occurs in the deployment phase of an attack [33], this would occur in the delivery and exploitation phase of the Cyber Kill Chain [5]. Evading detection at these phases means that ransomware can move to the destruction phases of attack, and this is where the victims would start to incur data loss. This shows that evading detection at these stages of attack means more success for the attacker in extorting the victim and should be the focus of AV/EDR tools to make sure ransomware cannot evade these stages using the techniques discussed.

Evasion of user-land function hooking is carried out by a few main techniques which are discussed below. It is worth noting that the effect of user-land hooking evasion is not just in the ability to evade the real-time interception capability of AV/EDR tools but also will have an effect on the quality of the data from dynamic analysis that is incorporated into the algorithms that classify malware as malicious or not.

In this section we will discuss the various techniques used for evasion of user-land hooking.

API Un-hooking

API un-hooking is used by a number of ransomware strains. Cizeron, n.d. included anti-API hooking as one of the mechanisms by which ransomware escape the notice of security tools.

In their analysis of the leaked source code from the Ransomware Group Conti, Alzahrani et al., (2022) describe the defence evasion tactic by which the ransomware disables function call hooking in an attempt to evade real time protection systems built into modern anti-virus and EDR software. As we can see from the example source in figure 10, each relevant Windows API library is loaded using the pLoadLibraryA function call.

If the library is loaded successfully, the code removes any function hooks (figure 11) from the library functions which results in disabling any ability for detection software to intercept API calls.

As outlined by Alzahrani et al., (2022) The removeHooks function achieves the unhooking by first detecting whether the first two bytes of the library loaded from disk differs from the in-memory version of the same library. If the byte differs this indicates that the function is hooked by the AV/EDR system. If the function is hooked, the ransomware replaces the in-memory function address with the original function address by reloading the library from disk and updating the in-memory address with the original memory address. This results in the removal of any function hooks from the AV/EDR system.

```
hKernel32 = pLoadLibraryA(OBFA("kernel32.dll"));
hWs2_32 = pLoadLibraryA(OBFA("ws2_32.dll"));
hAdvapi32 = pLoadLibraryA(OBFA("Advapi32.dll"));
hNtdll = pLoadLibraryA(OBFA("ntdll.dll"));
hRstrtmgr = pLoadLibraryA(OBFA("Rstrtmgr.dll"));
hOle32 = pLoadLibraryA(OBFA("Ole32.dll"));
hOleAut = pLoadLibraryA(OBFA("OleAut32.dll"));
hNetApi32 = pLoadLibraryA(OBFA("Netapi32.dll"));
hIphlp32 = pLoadLibraryA(OBFA("Iphlpapi.dll"));
hShlwapi = pLoadLibraryA(OBFA("Shlwapi.dll"));
hShell32 = pLoadLibraryA(OBFA("Shell32.dll"));
```

FIGURE 16 - DYNAMICALLY LOADING WINDOWS API LIBRARIES.

```
antihooks.h
1  #pragma once
2  #include <Windows.h>
3
4  VOID removeHooks(HMODULE hmodule);
```

FIGURE 17 - FUNCTION SIGNATURE FOR DISABLING FUNCTION HOOKS.

Rudimentary detection of API un-hooking can be obtained by hooking the following function calls to determine and block any API un-hooking activity.

LoadLibraryA, LoadLibraryExA, GetModuleFileNameA, CreateFile, CreateFileA, CreateFileMapping, CreateFileMappingA, MapViewOfFile, MapViewOfFileEx, VirtualProtect, VirtualProtectEx.

Looking for these function calls in sequence it may be possible to detect un-hooking. The problem with this is that different ransomware variants are likely to use different methods of unhooking.

There are variations of this evasion technique, notably the Whisper2Shout tool created by Bernardinetti et al. (2023) [47] bypasses the jmp trampoline function instead of replacing and overwriting the hooked function address by reloading it from disk. This technique serves to bypass any AV/EDR tool that can monitor its own function hooks.

Bernardinetti et al., (2023) [47] developed a technique called Whisper2Shout that evades AV software by un-hooking user land hooks without loading the library file from disk. Their novel approach was to avoid loading the library from disk as AV can detect this and flag it a suspicious behaviour. Their method also does not require kernel level access such is needed for syscall monitoring. Whisper2shout executes its technique with the capability to traverse pointers and retrieve memory region metadata only. Capabilities that are enabled by default for all Windows processes. Their work included analysis of 16 antivirus and 4 endpoint detection systems in which they defined a comparison of how API hooking is employed by each system.

In their work they mentioned that they successfully implemented un-hooking of the user land hooks employed by each AV/EDR system and was able to deliver the CobaltStrike beacon payload. However, no results have been published for this tool, so it is hard to evaluate whether this technique was effective.

Un-hooking techniques have been around for a while now. Analysis of the Conti source code utilising un-hooking goes back to 2022 [46]. Therefore AV/EDR have had time to build detection of un-hooking techniques into their systems by introducing countermeasures such as monitoring and protecting memory regions of executing processes. This is referenced in an article by Deep

Instinct in which it is mentioned that their agent software monitors for changes to specific memory regions to detect any un-hooking behaviour [48].

Direct syscall instantiation

As discussed in the "Windows API call vs syscall" section earlier, syscalls are executed by the native (NTDLL.DLL) libraries to temporarily transition the CPU into kernel mode to execute kernel land functions. Syscalls are instantiated by calling a specific ID related to the function to execute. These IDs are known to the NTDLL.DLL library and change with different versions of the Windows Operating system. Syscall IDs are easy to find and can be looked up dynamically as in the HookChain tool we will be using for our experiments [1]. There are also projects dedicated to tracking syscall IDs for the different versions of the Windows Operating System [49].

As AV/EDR tools hook the functions from the native NTLDLL.DLL library, instantiating a syscall directly is a way to circumvent these user-land function hooks [50]. Figure 16 demonstrates how this is carried out. The malware directly calls the specific syscall ID that corresponds to the function it wants to execute. If the malware utilizes this technique the AV/EDR tool is essentially blind to the malicious behaviour. This will also affect the dataset of any dynamic analysis relying on API monitoring as direct syscall will not be seen by the monitoring tool.

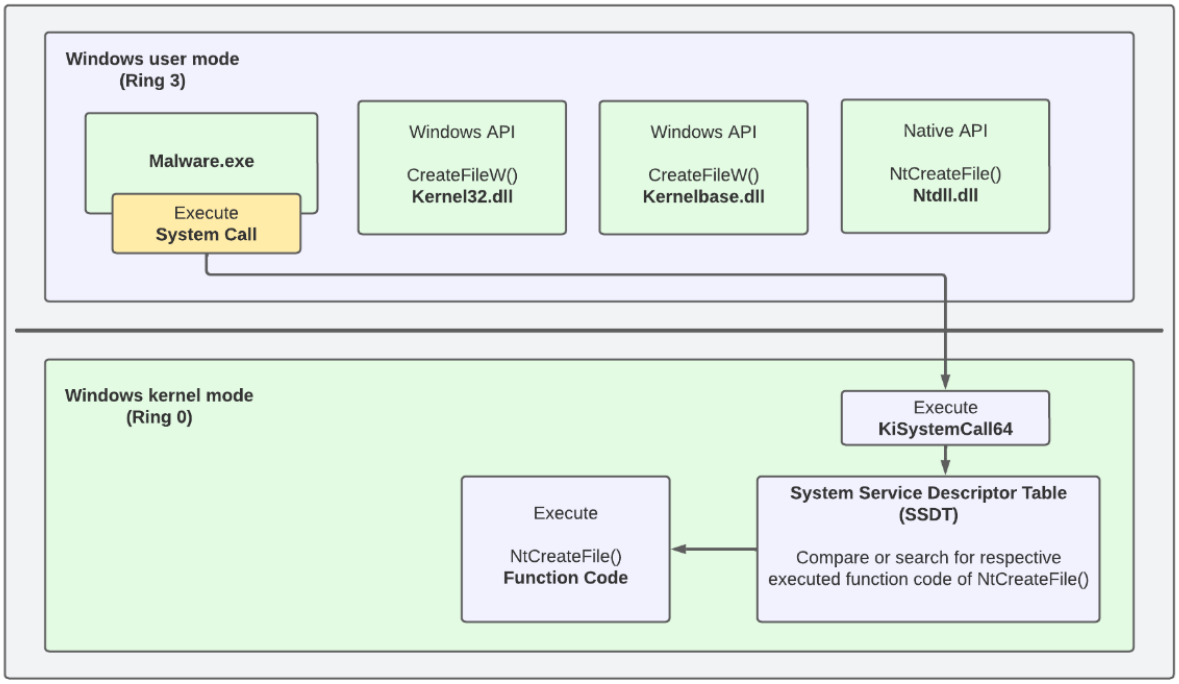


FIGURE 18 – FROM [50] EXAMPLE OF HOW DIRECT SYSCALLS WORK

Indirect syscall instantiation

There is an issue with the direct syscall technique described above in that the execution of the syscall takes place inside the memory area of the assembly containing the syscall. This is outside the normal usage of syscalls as it is expected that the syscall be made within the memory area of NTDLL.DLL only. This poses a problem for evasion as AV/EDR tools can check the memory location of the calling syscall assembly and flag an alert of the memory location is not the same at NTDLL.DLL.

The indirect syscall technique is primarily the same as the direct syscall technique but crucially in the indirect syscall technique the dynamic location for the memory address of the syscall instruction within NTDLL.DLL is used [51]. This process is shown in figure 19

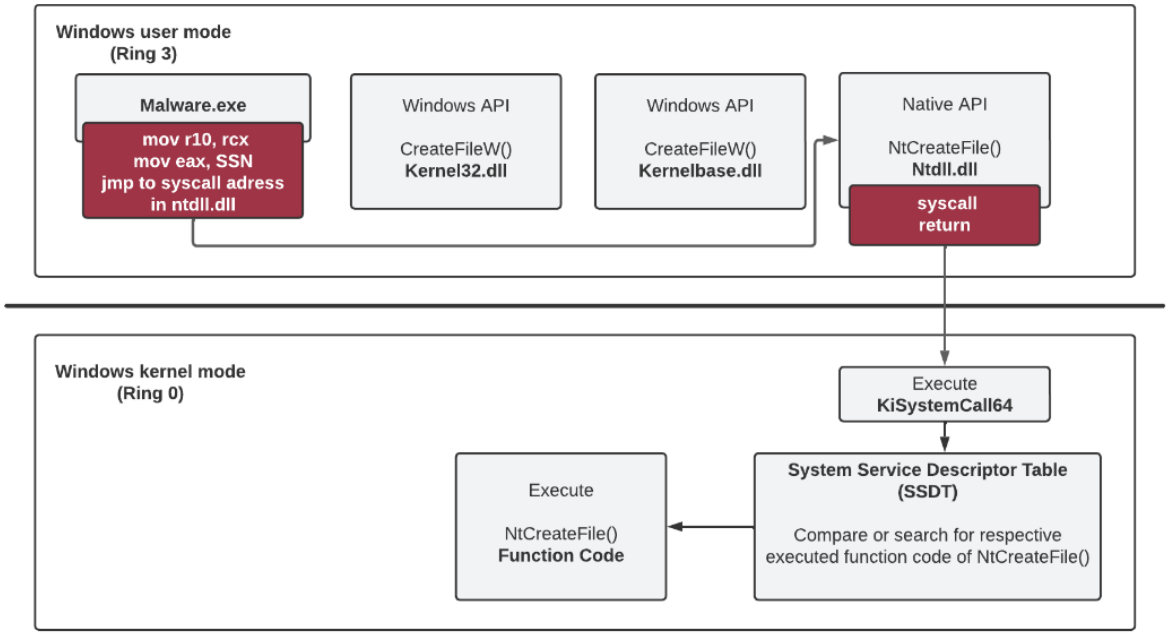


FIGURE 19 - INDIRECT SYSCALL TECHNIQUE [51]

This results in the JMP to the syscall seeming to come from NTDLL.DLL and removes the ability for the AV/EDR tool to detect rogue syscalls.

There are a number of implementations of indirect syscalls in the wild, namely the Syswhispers3 tool and the HookChain tool that this work uses. Considering its effectiveness outlined in the experiments from [1]and the fact that it is a relatively new technique, this work uses the technique of indirect syscalls in its experiments.

Process Injection

Process injection is the technique of using a candidate process already running on a machine to inject shellcode into its memory space for execution. One of the benefits of process injection is that it is a file-less technique, meaning that it can hide itself from the signature-based detection of any AV/EDR tool. The goal is to execute arbitrary code within the memory space of another process already running. Most AV/EDR tools that utilise function hooks inject their hooks at process creation time and inspect the process space for any malicious behaviour. Remote process injection reduces the risk of detection.

There are various ways to implement process injection techniques, but the HookChain tool used in this work utilises the technique of creating a thread in a remote process [1]. This increases the chance that any malicious payload will avoid the AV/EDR signature-based detection.

3.2 Ransomware evasion

In this section the use of hooking evasion techniques utilised in ransomware are discussed.

As described in the un-hooking section above, analysis of the leaked source code from the Conti Ransomware group tells us that un-hooking has been used by ransomware in the wild to evade detection [46].

In addition to this Olaimat M, et al (2021) in their analysis of ransomware evasion techniques, mentioned that current crypto-ransomware uses anti API hooking tactics to subvert the tracking

of API calls by AV/EDR systems. They also mention that current ransomware may sabotage AV/EDR attempts to track its behaviour by using non-OS API function calls altogether [52].

Malware loaders are often used in ransomware campaigns during the initial access phase of an attack. Going back to the Cyber Kill discussed in an earlier section [5] malware loaders fall into the delivery and exploitation phases of the attack. They aim to execute a second stage payload without being detected by AV/EDR.

Pikabot is a popular malware loader that is currently being used in malware campaigns. Pikabot is used to install backdoor payloads such as Cobalt Strike and Meterpreter [18] [19] onto compromised machines. Pikabot plays a significant role in the delivery and exploitation phases of an attack for ransomware campaigns such as BlackBasta.

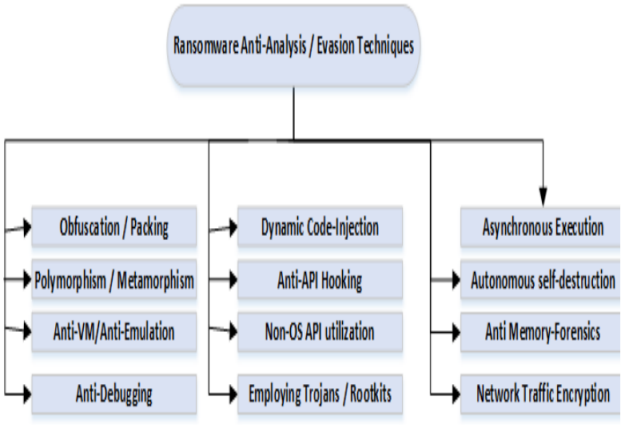


FIGURE 20 - RANSOMWARE EVASION TECHNIQUES

Non-OS API utilization and Anti-API Hooking are just two of many techniques that could be used by ransomware in each phases of its attack, There are various other techniques that could be utilised by ransomware, these are outlined in figure 19.

Pikabot analysis [53] [18] [19] has shown that it utilizes the indirect syscall technique mentioned in an earlier section.

BlackBasta ransomware is newer than the Conti ransomware and uses the more modern technique of indirect syscalls in contrast to the Conti technique of API unhooking. Considering this, the experiments in this work will focus on indirect syscalls over the other techniques discussed. This method is the most advanced and recent technique shown to be employed in recent ransomware campaigns to avoid user -land hooking detection and this technique has been shown to be effective against current AV/EDR systems. In the paper [1] there is evidence that indirect syscalls combined with process injection is effective against a number of AV/EDR tools. Combined with the fact that indirect syscalls are being used in ransomware campaigns, it would be useful to utilise this technique in the experiments

Conclusions

The conclusions of this literature review are discussed in this section, and these conclusions will be used to guide the research methodology and experimental setup.

It was found that ransomware can be broken down into various phases of attack that are identified by frameworks such as the Mitre Att&ck framework [17] In these frameworks various features can be identified for each phase of the ransomware attack, these features are formed by various techniques used by different malware variants. The techniques deemed useful for ransomware identification are the following: initial compromise, execution, defence evasion, lateral movement, command and control and impact techniques.

Different variants of ransomware sometimes use the same techniques for each phase of the attack. These techniques can be used to identify ransomware when it is executing on a host machine. Dynamic analysis of multiple ransomware samples can identify Windows API functions that would be useful in the detection of ransomware running on a host.

The literature has several techniques for use in ransomware identification based on dynamic analysis, the main one identified as the most useful is the use of the Windows API function call output to train machine learning classifiers. The patterns of function calls can be fed into heuristic or machine learning algorithms to be used to classify running processes as malicious or benign.

Once a detection algorithm has been created based on dynamic analysis of Windows API function calls, there needs to be some way for a detection system to intercept function calls to gather data and execute its classification algorithms. Most AV/EDR systems use a technique of hooking Windows API calls by injecting hooking code into a running process. This allows the AV/EDR system to watch the behaviour of running processes to determine whether they are malicious or not.

Ransomware has been found to employ AV/EDR evasion techniques that disable the ability of AV/EDR system to monitor its process. For example, Conti and BlackBasta have been found to use anti-hooking techniques to deliver their malicious payloads. These evasion techniques come in the form of API un-hooking, direct syscalls, indirect syscalls and process injection.

It is valuable to know which functions current AV/EDR systems use in their hooking strategies and which AV/EDR systems are vulnerable to these evasion techniques. There have been a couple of studies to identify how indirect syscalls are effective in evading AV/EDR functions [1] [2]. These studies have provided evidence that not all AV/EDR tools are using user-land function hooks in their defence strategy. For example, in [1] a technique was used to establish how many NTDLL.DLL functions are hooked by each AV/EDR tool. We replicate this experiment in this work to confirm and analyse the findings.

These conclusions have fulfilled the first aim of this work in which we sought to carry out a literature review of around the landscape of ransomware and how function hooking is utilized by AV/EDR tools as a defence layer to detect malicious behaviour of ransomware. The literature review included analysis of popular evasion techniques used in ransomware campaigns to deliver malicious payloads with the aim to avoid function hooking based detection.

6 Research Methodology Design

The experimental design is focused on the findings from previous studies [1] and [2] in which antivirus evasion techniques are employed to deliver a malicious payload to a victim's machine. This will identify whether function hooking based evasion techniques identified from the literature are still viable against modern versions of antivirus and endpoint detection software. This work expands on the previous studies by testing against newer versions of AV/EDR tools along with the OpenEDR endpoint detection software.

In the paper on the HookChain tool [1] the author creates a tool that implants indirect syscall redirections so that API calls bypass NTDLL.DLL and go direct to the kernel. This tool delivers and executes a payload via process injection and the paper tests various AV/EDR tools on their ability to detect and block the delivered payload. This work emulates the experiments from this paper by using the MessageBox and msfvenom payloads. These experiments test both a benign (MessageBox) and malicious (msfvenom) payloads and the experiments in this work do the same.

In addition to the above, this work combines the experiments from [2] which use a baseline shellcode runner that uses no evasion techniques to deliver and execute the shellcode.

This design gives us a good understanding of whether the evasion technique of indirect syscalls is effective against each AV/EDR tool.

This design will attempt to validate or invalidate the hypothesis that modern antivirus is still vulnerable to function hooking evasion, detail which function hooks each AV/EDR solution uses, and which AV/EDR tools are vulnerable. The experiments also match the hooked function calls to a sample malware dataset to gauge the extent to which evading function hooking may hide critical ransomware behaviour.

The experiments will focus on the most modern techniques that has been found to be used by current ransomware variants. The indirect syscall technique will be used along with the process injection technique in these experiments. The reason for this is that process injection serves to avoid the signature-based detection of the AV/EDR tool and the indirect syscall techniques hides the dynamic behaviour of the malware. If we just used the indirect syscall technique on its own then the signature based detection layer of the AV/EDR tool would alert on every run of our experiment if the shellcode signature matches the tools signature database. This means the experiments would never progress to the indirect syscall bypass phase.

Using a combination of these two techniques provides the most useful guide to whether AV/EDR tools that use user-land function hooking are still vulnerable to the technique of indirect syscalls.

6.1 Research Questions

The literature review identified that current ransomware is using evasion techniques to avoid the function hooking mechanisms of current AV/EDR tools. For example, the literature identifies anti-hooking techniques employed by the Conti ransomware variant and indirect syscalls used by the Pikabot malware loader used in the BlackBasta ransomware campaigns. The research questions in this work are focused on finding out to what extent function hooking evasion would impact the ability of AV/EDR tools to detect and defend against malicious ransomware behaviour.

Considering this, the research questions that this work attempts to answer are:

RQ1. Evaluate AV/EDR tools to identify whether they use function hooking and if so, what functions do they hook. This answer to this question will fulfil the second aim of this work.

RQ2. Evaluate current AV/EDR tools to identify which tools are vulnerable to the evasion techniques identified from the literature review This answer to this question will fulfil the third aim of this work.

6.2 Chosen AV/EDR tools

The chosen AV/EDR tools are based on the Gartner Magic Quadrant as of July 2024 [54]. The quadrant is based on a review of the AV/EDR tools currently in the market.



A lot of vendors provide their AV/EDR tools as one component of a suite of applications. Trial applications were obtained by applying for a free trial. However, not all vendors provided a trail application, so it was impossible to run experiments against these vendors tools.

This provides scope for further work expanding the tools tested to gain further insight into the landscape tools provided in the market.

6.1 Research Methodology

Research methods can fall into a few categories depending on the type of research carried out and the environment in which the experiment is conducted. Thomas W. Edgar and David O. Manz [55] provide a few types of research that can be carried out namely the categories of observational, mathematical and experimental research. Mathematical research is defined by a set of logic and formals proofs, this type of research wouldn't be appropriate for this work which falls into either the observational or experimental categories. Observational research occurs in a dynamic environment, where the environment cannot be controlled in a rigid manner. This work defines a controlled environment in which to conduct the experiments by using a well-defined sandbox that will be the same for each experiment conducted. In additional to this, this work defines independent and dependent variables from which we can control the experiment. The independent variables will be evasion techniques, and the AV/EDR tools used. The output evaluation will be based on two aspects: whether the payload was successfully delivered to the victim without detection and whether there was successful execution of the payload. Therefore, the dependent variable will be whether the payload was successfully detected and executed or not.

With all this in mind this work leans towards the experimental rather than observational type of research. In the realm of experimental research this work specifically takes the hypthetico-deductive approach [55] in which the hypothesis is that modern antivirus software is still vulnerable to function hooking evasion. Other related work has suggested this to be true and this work extends the idea to further prove the hypothesis against newer more modern antivirus and expands by including other techniques.

The first objective of this methodology is to gather information on which hooks each AV/EDR solution is using and map this to the tactics employed by ransomware.

A malware dataset obtained from [56] containing function calls executed by dynamic analysis of malware is used to match the hooked function calls to samples from the dataset. This identifies the extent to which malware might use these function calls to execute its functionality and shows how this functionality could be hidden from AV/EDR tools if hooking evasion was employed.

The steps executed to carry out the experiments are outlined below:

1. Install AV/EDR tool into Windows 11 Virtual Machine Sandbox.
2. Copy hookchain_finder64.exe into sandbox environment.
3. Execute hookchain_finder64.exe to get list of AV/EDR hooked functions.
4. Execute Indirect syscall evasion technique
    a. Copy HookChain_msg.exe to sandbox environment
    b. Execute HookChain_msg.exe
    c. Check AV/EDR tool for detection.

The following outputs will be determined by carrying out the steps above:

1. Which functions from NTDLL.DLL are hooked by each AV/EDR tool
2. Which evasion techniques both evade detection and allow execution of a payload.

Each hooked function identified from the hook finder tool from each execution against the set of AV/EDR tools is then searched for in the malware analysis dataset [56]. This identifies to what extent the function is used by real malware.

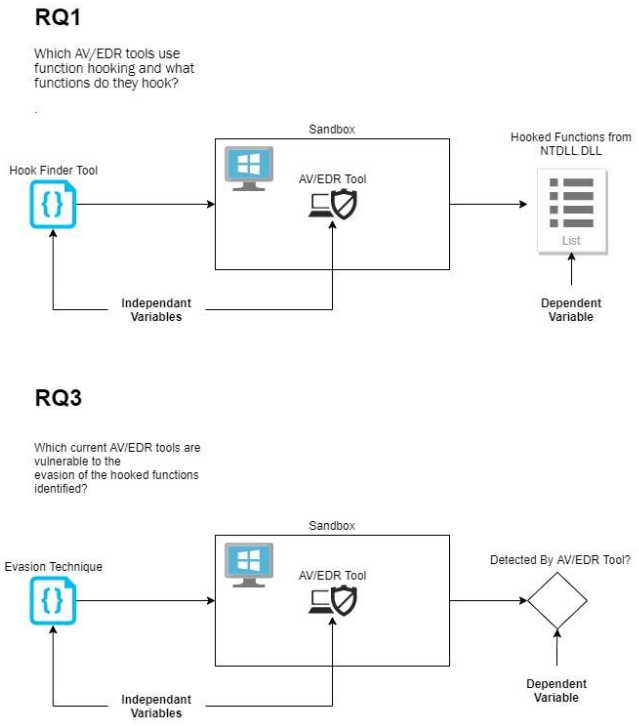


FIGURE 21 - STEPS TO CARRY OUT EXPERIMENTS

Evaluation of results

To understand the experiments the following equations tell us the probability of evasion and detection based on the outcome of execution of the malicious payload.

$$p_d = \frac{D}{X}$$

P_d = Probability of detection

Where D = detection of payload and X = tools tested.

$$p_e = \frac{E}{X}$$

P_e = Probability of execution

Where E = execution allowed and X = tools tested.

Tools used

HookChain

Protoreum Hook Checker

6.3 Experimental Environment Setup

The environment used for experiments are outlined below:

Host Machine

OS Name	Microsoft Windows 11 Pro
Version	10.0.22635 Build 22635
OS Manufacturer	Microsoft Corporation
System Name	DESKTOP-CVU31AQ
System Manufacturer	Micro-Star International Co., Ltd.
System Model	MS-7B48
System Type	x64-based PC
System SKU	Default string
Processor	Intel(R) Core(TM) i7-8700K CPU @ 3.70GHz, 3696 Mhz, 6 Core(s), 12 Logical Processor(s)
BIOS Version/Date	American Megatrends Inc. 2.54, 28/05/2018
SMBIOS Version	2.8
Embedded Controller Version	255.255
BIOS Mode	UEFI

Virtual Machine

OS Name	Microsoft Windows 11 Pro
Version	10.0.22000 Build 22000
OS Manufacturer	Microsoft Corporation
System Name	WIN11SANDBOX
System Manufacturer	Microsoft Corporation
System Model	Virtual Machine
System Type	x64-based PC
System SKU	None

Processor	Intel(R) Core(TM) i7-8700K CPU @ 3.70GHz, 3696 Mhz, 3 Core(s), 6 Logical Processor(s)
BIOS Version/Date	Microsoft Corporation Hyper-V UEFI Release v4.1, 06/04/2022
SMBIOS Version	3.1
Embedded Controller Version	255.255
BIOS Mode	UEFI

6.2 Experiments and Results

RQ1

The first experiment run is to answer research question 1. In this experiment the native Windows functions that are hooked by AV/EDR tools are identified by running a hook finder tool. This tool will output a list of native functions that are hooked by the AV/EDR tools tested.

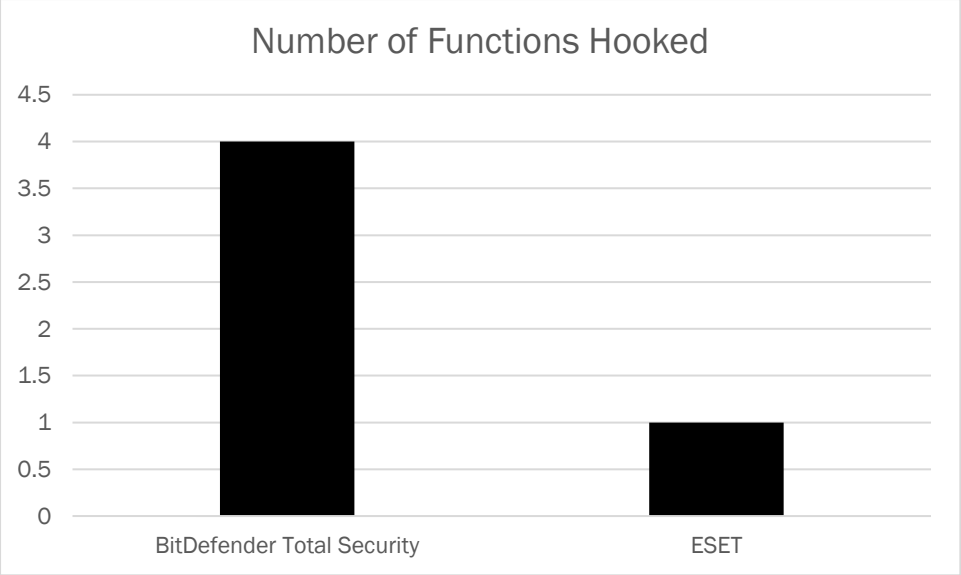
The list of functions is augmented with the number of times that function call is used by malware from the dynamic analysis dataset and from dynamic analysis of two Ransomware variants.

This experiment serves to identify to what extent these hooked functions are used by malware and ransomware, and which tools would theoretically be able to detect its behaviour.

This experiment also shows to what extent function hooking is used as a detection technique by current AV/EDR tools. It is worth noting that any function hooking evasion techniques used by malware and ransomware during its dynamic analysis would be omitted from the datasets, therefore it is not possible to see from this to what extent malware may be using function hooking evasion techniques. This experiment serves to show the extent to which AV/EDR tools could monitor the behaviour of malware should they hook the functions identified and by extension which tools may be vulnerable to evasion.

Hooked NTDLL.DLL Function	AV/EDR tools that are hooking the function
NtAdjustPrivilegesToken	OpenEDR, BitDefender
NtAlpcConnectPortEx	OpenEDR
NtAlpcConnectPort	Sophos, OpenEDR, BitDefender
NtAlpcCreatePort	OpenEDR, BitDefender
NtAlpcSendWaitReceivePort	OpenEDR, BitDefender
NtClose	OpenEDR, BitDefender
NtCommitTransaction	BitDefender
NtCreateMutant	BitDefender, OpenEDR
NtCreateProcess	BitDefender
NtCreateProcessEx	BitDefender, Sophos
NtCreateSection	OpenEDR, BitDefender
NtCreateSectionEx	BitDefender
NtCreateSymbolicLinkObject	BitDefender
NtCreateThread	OpenEDR, BitDefender
NtCreateThreadEx	OpenEDR, BitDefender
NtCreateTransaction	BitDefender
NtCreatUserProcess	BitDefender
NtDuplicateObject	BitDefender
NtLoadDriver	OpenEDR, BitDefender

NtMapViewOfSection	Sophos, BitDefender
NtMapViewOfSectionEx	Sophos, BitDefender
NtOpenProcess	BitDefender
NtQueryInformationProcess	BitDefender
NtQuerySystemEnvironmentValueEx	BitDefender
NtQueueApcThread	Sophos, BitDefender
NtQueueApcThreadEx	Sophos, BitDefender
NtQueueApcThreadEx2	Sophos
NtRaiseHardError	BitDefender
NtReadVirtualMemory	Sophos, OpenEDR, BitDefender
NtResumeThread	BitDefender
NtRollbackTransaction	BitDefender
NtSetContextThread	Sophos, BitDefender
NtSetInformationProcess	OpenEDR
NtSetInformationThread	OpenEDR, BitDefender
NtSetInformationTransaction	BitDefender
NtSetSystemEnvironmentValueEx	BitDefender
NtShutdownSystem	OpenEDR, BitDefender
NtSuspendThread	BitDefender
NtTerminateProcess	OpenEDR, BitDefender
NtUnmapViewOfSection	Sophos, BitDefender
NtWriteFile	BitDefender
NtWriteVirtualMemory	Sophos, OpenEDR, BitDefender
NtProtectVirtualMemory	Sophos
NtFreeVirtualMemory	Sophos
NtAllocateVirtualMemory	Sophos
NtAllocateVirtualMemoryEx	Sophos
NtQueryAttributesFile	Sophos
NtTerminateThread	OpenEDR
NtSystemDebugControl	OpenEDR
NtSetSystemInformation	OpenEDR
NtOpenSection	OpenEDR
NtOpenFile	OpenEDR
NtMakeTemporaryObject	OpenEDR
NtDeviceIoControlFile	OpenEDR
NtCreateSemaphore	OpenEDR
NtCreateSymbolicLinkObject	OpenEDR
NtCreatePort	OpenEDR
NtCreateFile	OpenEDR
NtCreateEvent	OpenEDR
NtCreateEventPair	OpenEDR
NtConnectPort	OpenEDR



Result

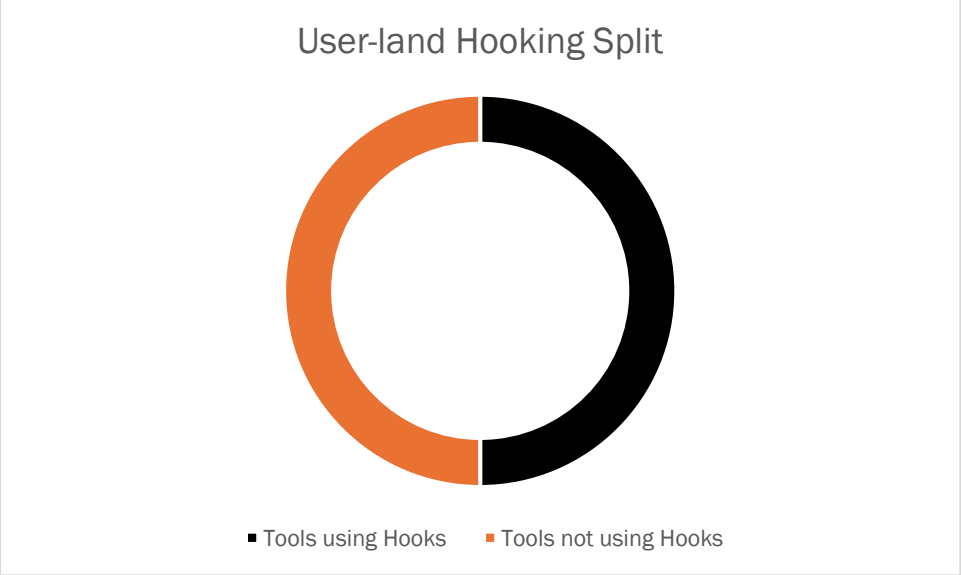
The above analysis shows that some AV/EDR vendors opt for many user-land function hooks, for example Sophos and OpenEDR. In contrast other vendors opt for no hooks at all, for example, Windows Defender and ESET.

This shows that there are still some AV/EDR vendors relying on user-land hooks for detection or at least for behaviour monitoring. The AV/EDR tools using a higher number of user-land hooks would be more vulnerable to the evasion techniques discussed in this work. This is because for the AV/EDR tool to implement many user-land function hooks, which take a system performance hit, they must be relying on those hooks to some extent for their detection mechanisms.

The tools using no user-land hooks by contrast would be relying on other methods of detection and would be less affected by the evasion techniques discussed.

The tools with the most user-land hooks have been ranked in the table below. A comparison has been made to previous papers that also made measurements of user-land hooks for each tool. The percentage change indicates whether user-land hooks have increased or decreased since last tested.

Tool	Current User-land hooks	Previous User-land hooks	% change
BitDefender Total Security	39	39 [1]	0%
OpenEDR	31	7 [2]	342%
Sophos	16	13 [1]	23%
ESET Advanced Protection	0	0 [1]	0%
Microsoft Windows Defender	0	0 [1]	0%
Cisco Secure Endpoint	0	n/a	0%



As we can see from the analysis above, several current versions of the AV/EDR tools hook native NTDLL.DLL functions to carry out it's real time analysis. Around 50% of AV/EDR vendors utilize function hooking as part of their defence layers. This result is in line with the findings from [1] which show a similar result

This result leads us to believe that evasion techniques that can evade the user-land function hooks will have an advantage over AV/EDR tools that use function hooking as part of their defence. This is because its behaviour cannot be detected by the AV/EDR tool and therefore could freely execute its malicious activities. The experiments carried out to answer RQ2 aim to confirm this hypothesis.

Currently any API function calls used by malware would be detected by AV/EDR tools if the tool utilises function hooking. However, as identified by the literature, ransomware variants such as the Conti variant and BlackBasta with the Pikabot malware loader are using evasion techniques in their source code, specifically API un-hooking and indirect syscalls respectively. If malware currently using the hooked functions were to employ the techniques identified in this work, they would be able to hide their malicious intent from the AV/EDR tools.

The answer to this research question shows the potential for evading AV/EDR by evasion of user-land function hooks. This is confirmed from the literature showing current malware variants such as Conti and BlackBasta are utilizing these techniques in the wild. This identifies the need for further analysis of which tools are vulnerable to function hooking evasion to help understand how much impact function hooking evasion would have on endpoint detection software.

It is worth nothing that from the answer to RQ1 it also shows that 50% of AV/EDR vendors are not using function hooking as part of their defence layers and it is unclear from the literature whether this is a trend to move away from function hooking as a defence technique. A move away from function hooking could possibly be due to evasion techniques becoming more pervasive in malware and no longer worth the performance cost. This finding is aligned with Lewis & Rimal, (2024) [2] who mentions that not all AV/EDR vendors use extensive user-land hooking for performance reasons.

A comparison was made in table 1 of the difference between user-land hooks in previous studies where the number of user-land hooks were measured, such as in [1] and [2]. It was found that some vendors that utilised function hooks had an increase in the number of hooks identified in the newer version. Namely these were the OpenEDR and Sophos tools that increased their user-land hooks by 345% and 23% respectively. This however is not enough data to conclusively say

whether function hooking is being more or less utilised broadly across vendors. More data would be needed to achieve this.

It would be interesting as future work to gather further evidence either confirm or deny a move away from function hooking as a defence mechanism.

In the next section the experiments for RQ2 are carried out against each AV/EDR tool with the evasion techniques identified to see which AV/EDR tools are vulnerable.

RQ2

Baseline experiment

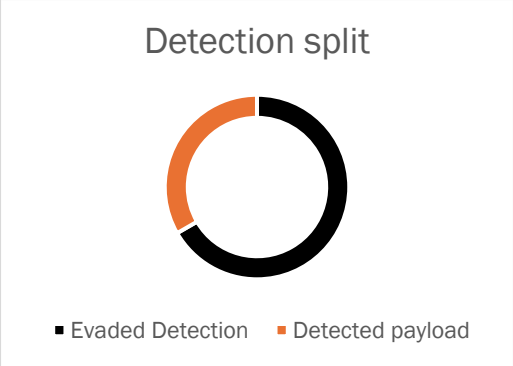
In this experiment a simple shellcode loader tool was written in c to execute a shellcode payload. This experiment serves to identify if the same msfvenom calc.exe payload is detected without any evasion techniques employed.

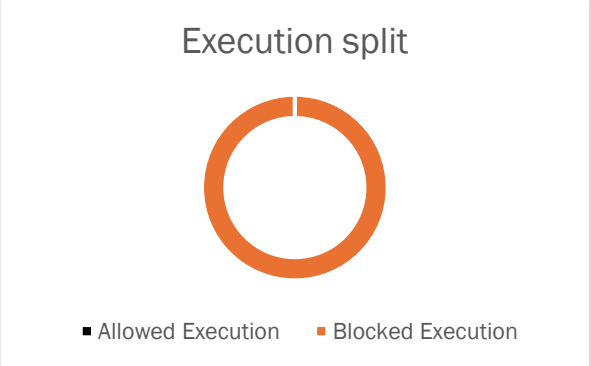
This baseline experiment is based on the baseline experiments from

If the AV/EDR tool tested identified the payload as malicious in the baseline test but subsequently allows execution of the payload using an evasion technique, this signifies that the evasion technique would be successful in delivering and executing malicious behaviour without detection.

Results

Product	Version	Loader	Payload	Evaded Detection	Allowed Execution
Microsoft Windows Defender	4.182205.7	Baseline wrapper	Msvenom calc.exe	✗	✗
ESET Advanced Protection	11.1.2052.0	Baseline wrapper	Msvenom calc.exe	✓	✗
Sophos	2024.2.4.1.0	Baseline wrapper	Msvenom calc.exe	✓	✗
BitDefender Total Security	27.0.42.214	Baseline wrapper	Msvenom calc.exe	✗	✗
OpenEDR	13.2.0.9560	Baseline wrapper	Msvenom calc.exe	✓	✗
Cisco Secure Endpoint	8.4.2.30317	Baseline wrapper	Msvenom calc.exe	✓	✗





If we plug the result above into our probability equation, then we see that

$$p_d = \frac{D}{X}$$

$$p_d = \frac{2}{6} = 0.333\ldots$$

There is a 33% chance of detection when using the baseline shellcode loader.

$$p_e = \frac{E}{X}$$

$$p_e = \frac{0}{6} = 0$$

There is a 0% chance of evasion when using the baseline shellcode loader.

Windows Defender
Execution of the baseline shellcode loader with the msfvenom payload on Windows Defender resulted in detection and quarantine of the shellcode wrapper executable.

ESET Advanced Security
Execution of the baseline shellcode loader with the msfvenom payload on ESET Advanced Security resulted in no detection but also no execution of the shellcode wrapper executable.

Sophos
Execution of the baseline shellcode loader with the msfvenom payload on ESET Advanced Security resulted in no detection but also no execution of the shellcode wrapper executable.

BitDefender Total Security
Execution of the baseline shellcode loader with the msfvenom payload on Windows Defender resulted in detection and quarantine of the shellcode wrapper executable.

OpenEDR (Comodo)
Execution of the baseline shellcode loader with the msfvenom payload on OpenEDR resulted in no detection but also no execution of the shellcode wrapper executable.

Cisco Secure Endpoint
Execution of the baseline shellcode loader with the msfvenom payload on Cisco Secure Endpoint resulted in no detection but also no execution of the shellcode wrapper executable.

Indirect syscall experiments

To answer the question of which AV/EDR products are vulnerable, the HookChain tool is used [1] to execute the technique of indirect syscalls to bypass AV/EDR function hooks in NTDLL.DLL.

This work is based on previous experiments from [1] which show that indirect syscalls are a valid method of delivering a payload to a victim's machine and execution malicious functions.

The HookChain tool delivers a payload into an existing process running on the machine using the technique of process injection which was explained earlier in this work. The first payload used is a simple message box that is injected into the notepad.exe application. The payload is injected as shell code into an existing running process and uses functions from NTDLL.DLL. This shows that if the hooks have been evaded successfully any behaviour from the injected shellcode would be invisible to the AV/EDR tool.

In the first experiment a simple payload is used that executes a Windows API function from user32.dll to avoid the fact that injecting a payload with a known signature would trigger other detection techniques employed by the AV/EDR tool. The AV/EDR tool could detect the shell code embedded in the PE file and trigger an alert. Executing a simple payload avoids this signature-based detection and allows focus to be put on the indirect syscall technique delivered by remote process injection. This is the same experiment as carried out in the first experiments from [1].

A second payload was then used created using the msvenom tool. This payload delivers the meterpreter tool which loads the calc.exe program. Delivering the meterpreter payload means that the AV/EDR tool may pick up on the shellcode signature before it has a chance to execute its functions using indirect syscalls. This can be seen in the baseline experiment where the meterpreter payload is wrapped in a simple executable with no evasion techniques. If the HookChain loader successfully delivers the payload then it is an indication that the indirect syscall combined with remote process injection technique is bypassing both the signature-based layer and the behavioural based layer of the AV/EDR tool. This payload experiment serves to show that by hiding the payload using process injection we get to see if that maclious behaviour of the payload in the form of meterpreter opening another process is hidden from the AV/EDR tool by implementing the indirect syscall evasion technique. This is similar to the second experiments from [1] however a different payload was used due to the fact that the source code for the second payload was unavailable for reference.

Results

Windows Defender

Execution of the message box payload using the HookChain loader resulted in no detection and full execution of the shellcode.



FIGURE 22 - HOOKCHAIN TOOL DELIVERING MESSAGE BOX PAYLOAD

Execution of the msfvenom payload using the HookChain loader resulted in no detection and full execution of the shellcode.

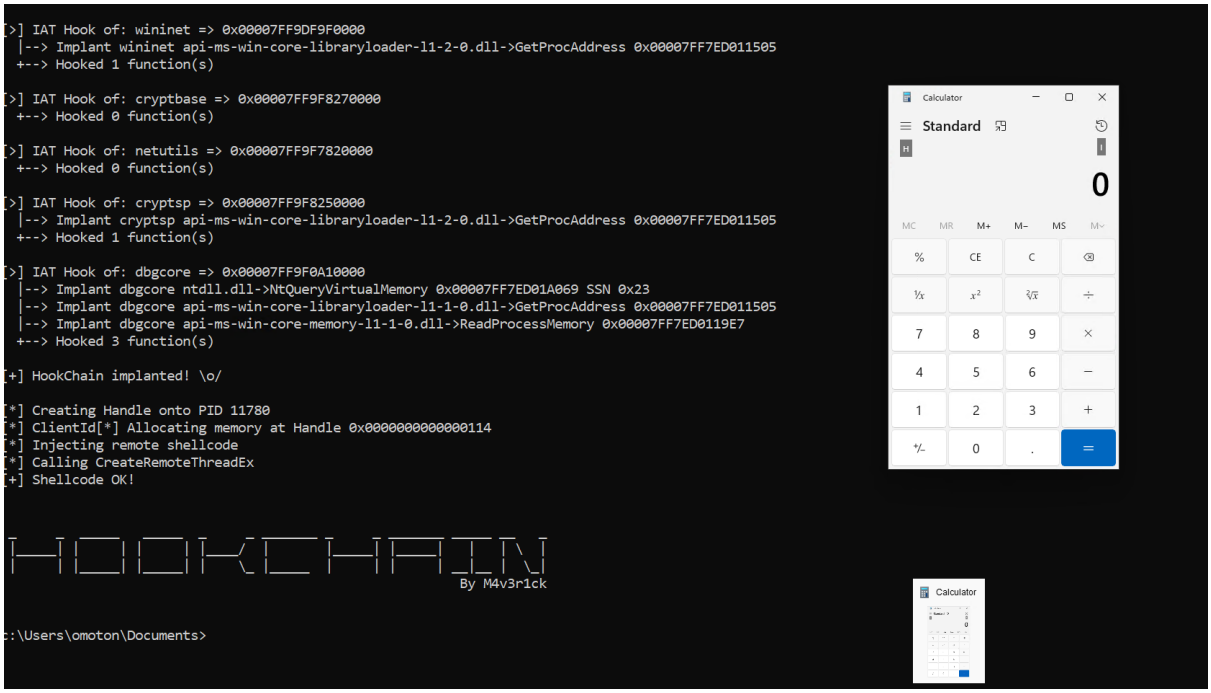
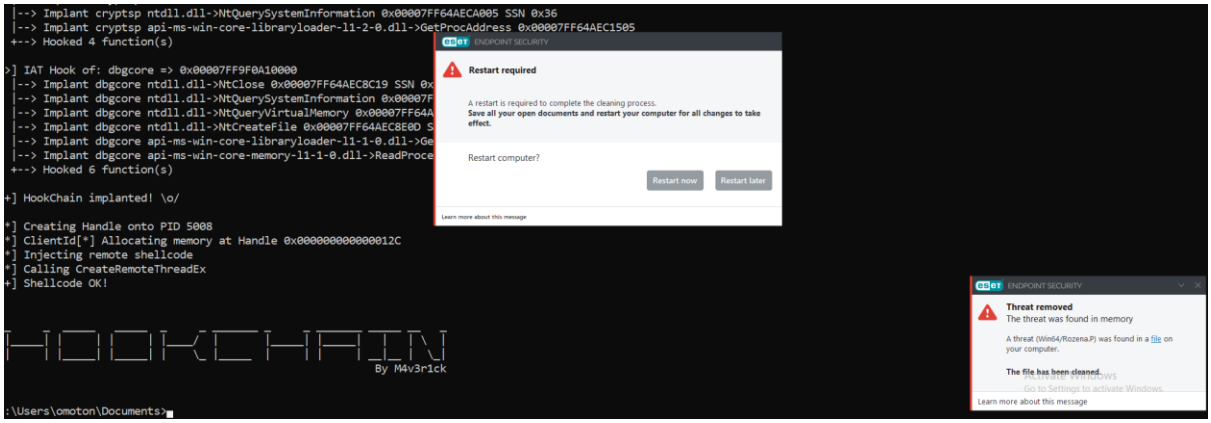


FIGURE 23 - EXECUTION OF MSFVENOM CALC.EXE ON WINDOWS DEFENDER

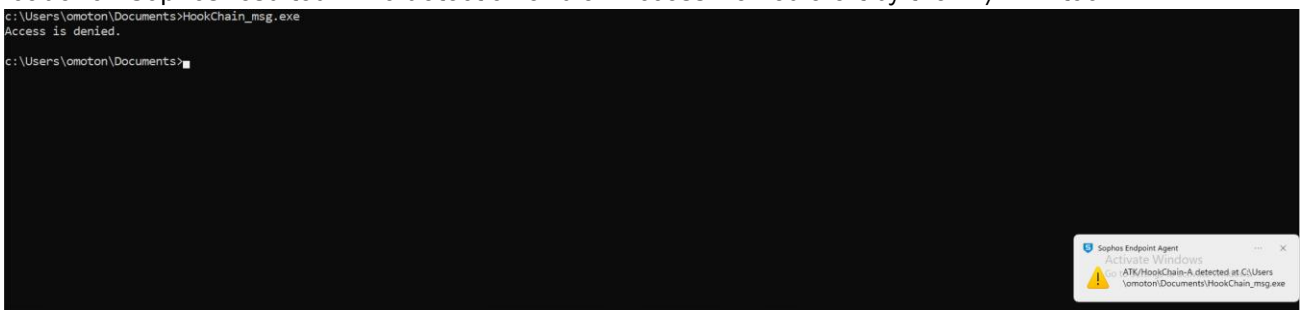
ESET Advanced Security

Execution of both the message box payload and the msfvenom payload using the HookChain loader on ESET Advanced Security resulted in no detection and a threat removed action by the AV/EDR tool.



Sophos

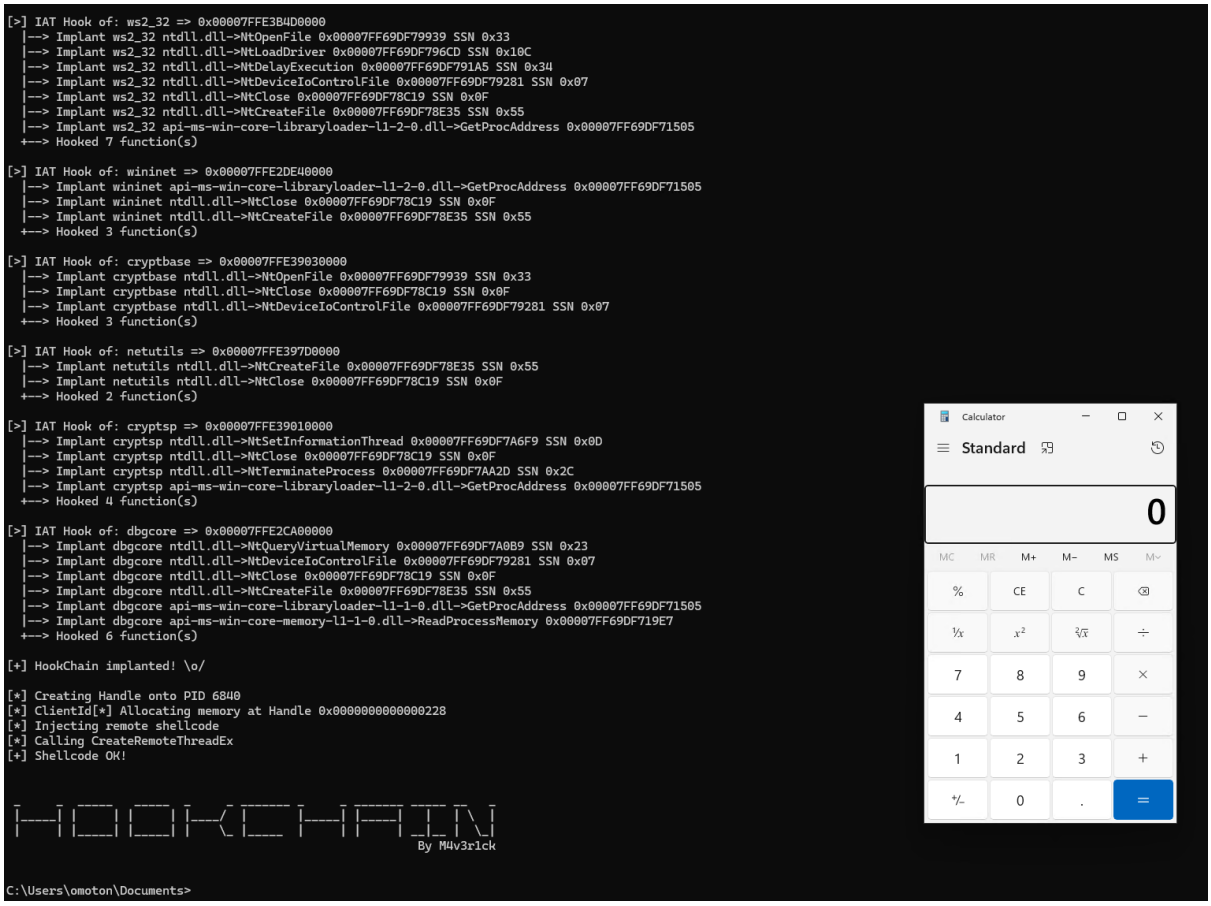
Execution of both the message box payload and the msfvenom payload using the HookChain loader on Sophos resulted in no detection and an Access Denied alert by the AV/EDR tool.



OpenEDR

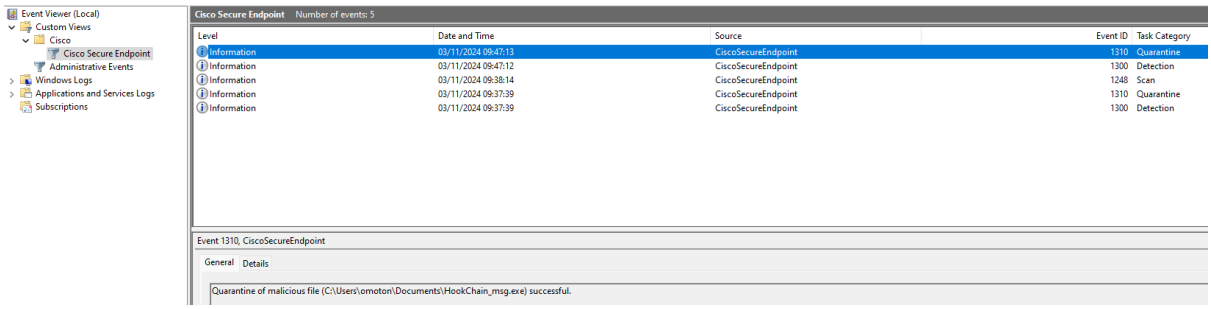
Execution of the message box payload using the HookChain loader resulted in no message box being displayed and notepad process crashing.

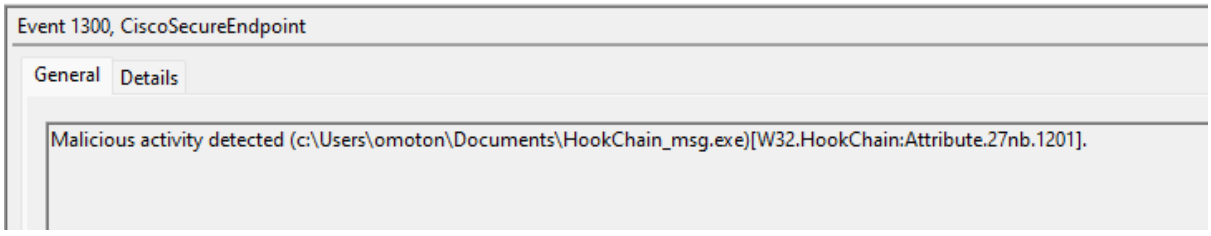
Execution of the msfvenom payload using the HookChain loader resulted in no detection and full execution of the shellcode.



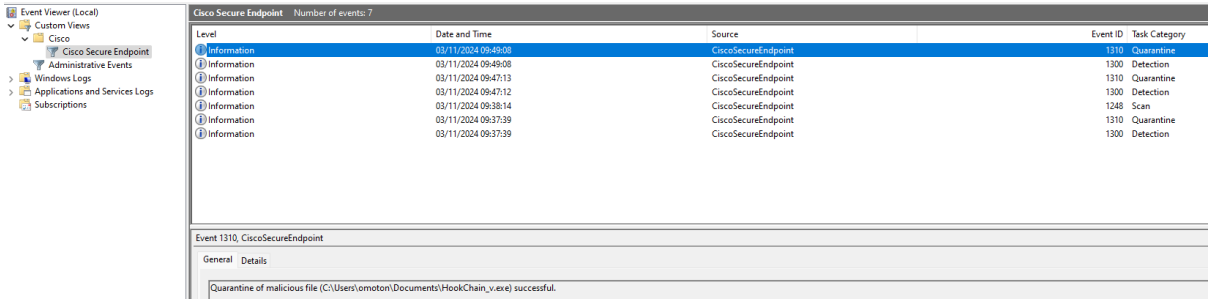
Cisco Secure Endpoint

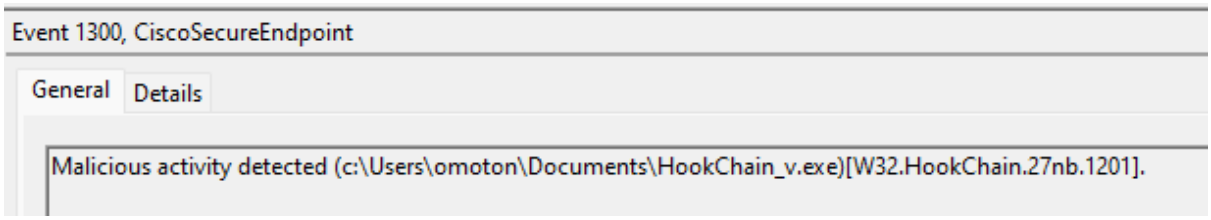
Execution of the message box payload using the HookChain loader resulted in blocked execution of the executable for malicious activity and quarantine by the AV/EDR tool.



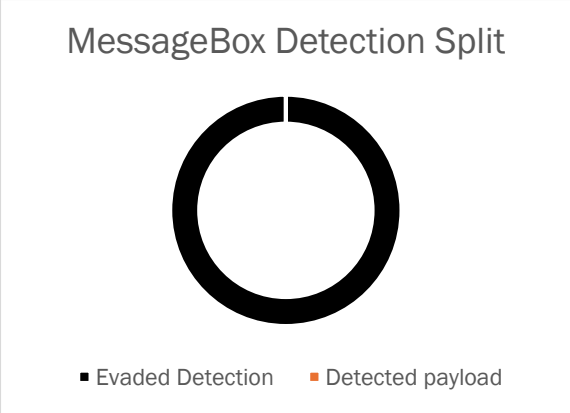


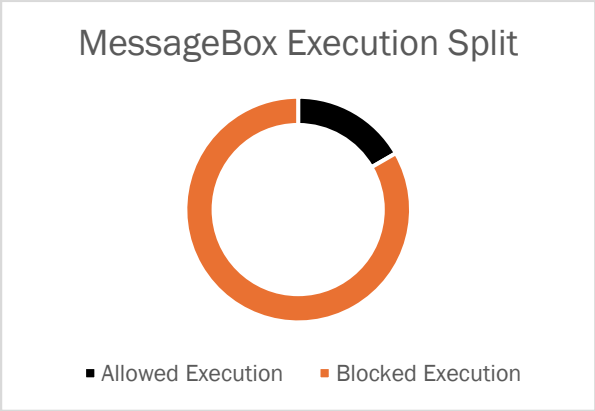
Execution of the msfvenom payload using the HookChain loader resulted in blocked execution of the executable for malicious activity and quarantine by the AV/EDR tool.

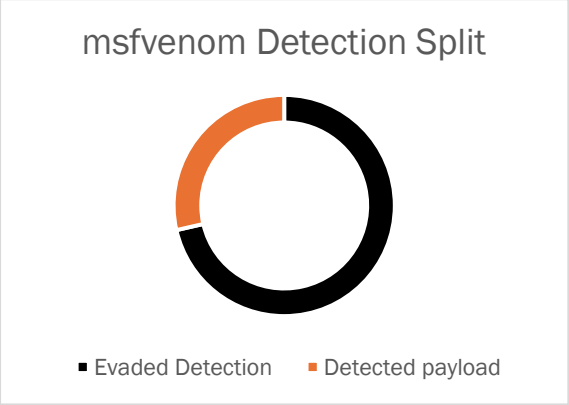


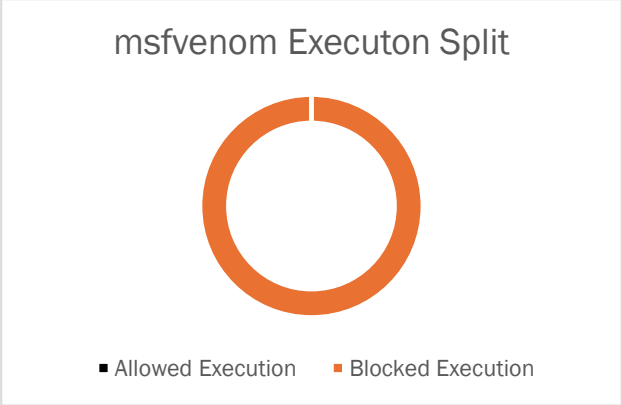


Product	Version	Loader	Payload	Number of hooked functions	Evaded Detection	Allowed Execution
Microsoft Windows Defender	4.182205.7	HookChain	MessageBox Shellcode	0	✓	✓
Microsoft Windows Defender	4.182205.7	HookChain	Msvenom calc.exe	0	✓	✓
ESET Advanced Protection	11.1.2052.0	HookChain	MessageBox Shellcode	0	✓	✗
ESET Advanced Protection	11.1.2052.0	HookChain	Msvenom calc.exe	0	✓	✗
Sophos	2024.2.4.1.0	HookChain	MessageBox Shellcode	16	✓	✗
Sophos	2024.2.4.1.0	HookChain	Msvenom calc.exe	16	✓	✗
OpenEDR	13.2.0.9560	HookChain	MessageBox Shellcode	31	✓	✗
OpenEDR	13.2.0.9560	HookChain	Msvenom calc.exe	31	✓	✓
Cisco Secure Endpoint	8.4.2.30317	HookChain	MessageBox Shellcode	0	✓	✗
Cisco Secure Endpoint	8.4.2.30317	HookChain	Msvenom calc.exe	0	✓	✗









If we plug the results above into our probability equations, then we see that

$$p_d = \frac{D}{X}$$

$$p_d = \frac{0}{6} = 0$$

There is a 0% chance of detection when using indirect syscalls and process injection with the message box shellcode.

$$p_e = \frac{E}{X}$$

$$p_e = \frac{1}{6} = 0.166\ldots$$

There is a 16% chance of execution when using indirect syscalls and process injection with the message box shellcode.

$$p_d = \frac{D}{X}$$

$$p_d = \frac{0}{6} = 0$$

There is a 0% chance of detection when using indirect syscalls and process injection with the msfvenom shellcode.

$$p_e = \frac{E}{X}$$

$$p_e = \frac{2}{6} = 0.333 \ldots$$

There is a 33% chance of execution when using indirect syscalls and process injection with the msfvenom shellcode.

Experiment	Chance of detection	Chance of execution
Baseline msfvenom shellcode	33%	0%
Indirect syscall with message box payload	0%	16%
Indirect syscall with msfvenom payload	0%	33%

It is evident from the results above that without evasion techniques employed by the malware, there is a much higher chance of both detection and the blocking of execution of the payloads from the AV/EDR tool.

Using the methods of indirect syscalls and process injection the chances of detection fell to 0% which means that the combination of these techniques is enough to make the AV/EDR tools blind to malicious payloads. In terms of detection, it is unclear from these experiments what caused the drop in detection rate, but as process injection is intended to hide the payload from detection, it is most likely this technique that achieves the drop in detection rate.

In terms of execution of the malicious payload, specifically the msfvenom generated meterpreter payload the chance of execution increases greatly when employing the indirect syscall technique. This shows that indirect syscalls are effective against some a large proportion of AV/EDR tools tested.

Discussion of Results

We can see from the answer to RQ1 that not all AV/EDR vendors utilize function hooking in their defence mechanisms. Windows Defender is a notable case, as you can see from the number of functions hooked it doesn't seem like user-land function hooking defence layer for used extensively if at all. Evasion techniques such as indirect syscalls would make no difference to the evasion capabilities against Windows Defender as it would rely on other methods of detection.

That said Windows Defender was notable for allowing execution of the msfvenom generated payload which means the Hookchain tool was able to evade detection either through its use of remote process injection or indirect syscalls. As Windows Defender doesn't seem to hook any user-land functions then it was most likely that the remote process injection technique allowed evasion.

This contrasts with other tools like the ESET tool which also lacked any user-land function hooks. However, it was able to detect the msfvenom payload, mostly like due to other detection techniques such as memory monitoring.

The OpenEDR client security solution was interesting in that it demonstrated a higher number of user-land function hooks. Executing the msfvenom shellcode without any evasion technique being employed resulted in the app crashing without execution of the payload. Executing the msfvenom payload with the indirect syscall technique using HookChain resulted in full execution

of the payload. This indicates that the OpenEDR tool puts more emphasis on user land hooks to detect process behaviour and as such the HookChain tool was able to bypass these user land hooks and execute the msfvenom shellcode.

The Sophos tool also used extensive user-land hooks and as such was able to detect the msfvenom payload being executed. The alert message generated suggest that the malicious payload is detected in memory and this detection may have been due to a function hook intercepting writes to virtual memory space. For example, we know that the HookChain tool makes use of the NtWriteVirutalMemory function which the Sophos tool hooks. This function call should be replaced with an indirect syscall in the HookChain tool, but Sophos was still able to detect it. Due to the blackbox nature of the AV/EDR tool it is unclear exactly how the Sophos tool was able to detect the malicious payload in memory, but this points to a layered defence approach such that if the NtWriteVirtualMemory function hook was bypassed, another layer of defence may have detected the shellcode being executed.

The conclusion to be made from the experiments are that AV/EDR vendors rely heavily on a layered defence for their detection mechanisms. This is shown by the fact that some tools are able to detect malicious payloads even when using user-land hook evasion techniques such as indirect syscalls. This agrees with the findings from [1] which found that the indirect syscall technique was not always able to defeat all AV/EDR tools.

AV/EDR vendors that put more focus on user land hooks are more vulnerable to evasion techniques. This is demonstrated by the OpenEDR tool being unable to detect the msfvenom shellcode when indirect syscall are used. This is also in agreement with [1] who found that the tools with the most hooks were vulnerable to the indirect syscall technique.

The indirect syscall technique is the most modern evasion technique in the literature and its capability to execute malicious payloads is high against a large portion of AV/EDR tools, especially if they implement a higher number of user-land function hooks.

Aims and Objectives

In this section the original aims and objectives of this work are evaluated to determine its level of success in achieving those aims and objectives.

The primary goal of this work was to evaluate antivirus and endpoint detection systems to ascertain to what extent they use function hooking within their layers of defence and to what extent function hooking evasion techniques work against the current version of those tools.

The original aims and objectives are specified here:

Aim one: Carry out literature review of ransomware behaviour and the threat landscape related to AV/EDR evasion, focusing on Windows API function hooking as a defence technique.

Aim two: From the results of the literature review, evaluate function hooking in current AV/EDR systems to determine the extent of use in each tool.

Aim three: Carry out experiments to evaluate the level of defence in each AV/EDR tool when it comes to detection of malicious payloads.

Aim Four: Provide analysis of experiment results to answer the research questions proposed.

Aim One – Literature Review

Ransomware is

References

[1] H. M. Benedito Dias de Carvalho Junior aka and ck Sec, “HookChain: A new perspective for Bypassing EDR Solutions,” Apr. 2024, Accessed: Sep. 30, 2024. [Online]. Available: https://arxiv.org/abs/2404.16856v3

[2] T. M. Lewis and B. P. Rimal, “Effects of Removing User-Land Hooks in Endpoint Protection During Attack Experiments,” *IEEE Access*, vol. 12, pp. 15820–15844, 2024, doi: 10.1109/ACCESS.2024.3357525.

[3] T. McIntosh, A. S. M. Kayes, Y. P. P. Chen, A. Ng, and P. Watters, “Ransomware Mitigation in the Modern Era: A Comprehensive Review, Research Challenges, and Future Directions,” *ACM Computing Surveys (CSUR)*, vol. 54, no. 9, Oct. 2021, doi: 10.1145/3479393.

[4] A. Liska and T. Gallo, “Ransomware: Defending Against Digital Extortion - Allan Liska, Timothy Gallo - Google Books.” Accessed: Jul. 09, 2024. [Online]. Available: https://books.google.co.uk/books?hl=en&lr=&id=IIORDQAAQBAJ&oi=fnd&pg=PR2&ots=EwogDC50hc&sig=KR4atDuGyCaMIteD9q6KUjX2xrg&redir_esc=y#v=onepage&q&f=false

[5] T. Dargahi, A. Dehghantanha, P. N. Bahrami, M. Conti, G. Bianchi, and L. Benedetto, “A Cyber-Kill-Chain based taxonomy of crypto-ransomware features,” *Journal of Computer Virology and Hacking Techniques*, vol. 15, no. 4, pp. 277–305, Dec. 2019, doi: 10.1007/S11416-019-00338-7/TABLES/5.

[6] J. Hernandez-Castro, A. Cartwright, and E. Cartwright, “An economic analysis of ransomware and its welfare consequences,” *R Soc Open Sci*, vol. 7, no. 3, Mar. 2020, doi: 10.1098/RSOS.190023.

[7] J. Ispahany, R. Islam, Z. Islam, and M. Arif Khan, “Ransomware Detection Using Machine Learning: A Review, Research Limitations and Future Directions; Ransomware Detection Using Machine Learning: A Review, Research Limitations and Future Directions,” *IEEE Access*, vol. 12, 2024, doi: 10.1109/ACCESS.2024.3397921.

[8] A. Kapoor, A. Gupta, R. Gupta, S. Tanwar, G. Sharma, and I. E. Davidson, “Ransomware Detection, Avoidance, and Mitigation Scheme: A Review and Future Directions,” *Sustainability 2022, Vol. 14, Page 8*, vol. 14, no. 1, p. 8, Dec. 2021, doi: 10.3390/SU14010008.

[9] P. Mohan Anand, P. V. Sai Charan, and S. K. Shukla, “A Comprehensive API Call Analysis for Detecting Windows-Based Ransomware,” *Proceedings of the 2022 IEEE International Conference on Cyber Security and Resilience, CSR 2022*, pp. 337–344, 2022, doi: 10.1109/CSR54599.2022.9850320.

[10] P. Maniriho, A. N. Mahmood, and M. J. M. Chowdhury, “API-MalDetect: Automated malware detection framework for windows based on API calls and deep learning techniques,” *Journal of Network and Computer Applications*, vol. 218, p. 103704, Sep. 2023, doi: 10.1016/J.JNCA.2023.103704.

[11] S. Sheen and A. Yadav, “Ransomware detection by mining API call usage,” *2018 International Conference on Advances in Computing, Communications and Informatics, ICACCI 2018*, pp. 983–987, Nov. 2018, doi: 10.1109/ICACCI.2018.8554938.

[12] N. Yadav, G. Kaur, S. Kaur, A. Vashisth, and C. Rohith, "A Complete Study on Malware Types and Detecting Ransomware Using API Calls," *2021 9th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions), ICRITO 2021*, 2021, doi: 10.1109/ICRITO51393.2021.9596085.

[13] M. Hirano and R. Kobayashi, "Machine Learning Based Ransomware Detection Using Storage Access Patterns Obtained from Live-forensic Hypervisor," *2019 6th International Conference on Internet of Things: Systems, Management and Security, IOTSMS 2019*, Oct. 2019, doi: 10.1109/IOTSMS48152.2019.8939214.

[14] S. H. Kok, A. Abdullah, N. Z. Jhanjhi, and M. Supramaniam, "Prevention of Crypto-Ransomware Using a Pre-Encryption Detection Algorithm," *Computers 2019, Vol. 8, Page 79*, vol. 8, no. 4, p. 79, Nov. 2019, doi: 10.3390/COMPUTERS8040079.

[15] "The Antivirus Hacker's Handbook - Joxean Koret, Elias Bachaalany - Google Books." Accessed: Aug. 11, 2024. [Online]. Available: https://books.google.co.uk/books?hl=en&lr=&id=wqV1CgAAQBAJ&oi=fnd&pg=PR19&dq=antivirus+hackers+handbook&ots=HI3koCn1vL&sig=Yle-4qJVzj1OaWkWzlhQPJhK7bA&redir_esc=y#v=onepage&q=antivirus%20hackers%20handbook&f=false

[16] K. Begovic, A. Al-Ali, and Q. Malluhi, "Cryptographic ransomware encryption detection: Survey," *Comput Secur*, vol. 132, p. 103349, Sep. 2023, doi: 10.1016/J.COSE.2023.103349.

[17] "MITRE ATT&CK®." Accessed: Jul. 21, 2024. [Online]. Available: https://attack.mitre.org/

[18] "Black Basta-Affiliated Water Curupira's Pikabot Spam Campaign | Trend Micro (UK)." Accessed: Oct. 25, 2024. [Online]. Available: https://www.trendmicro.com/en_gb/research/24/a/a-look-into-pikabot-spam-wave-campaign.html

[19] "Pikabot: Battling a Fast-Moving Loader Malware | Darktrace Blog." Accessed: Nov. 02, 2024. [Online]. Available: https://darktrace.com/blog/pikabot-malware-battling-a-fast-moving-loader-malware-in-the-wild

[20] "Native API, Technique T1106 - Enterprise | MITRE ATT&CK®." Accessed: Jul. 21, 2024. [Online]. Available: https://attack.mitre.org/techniques/T1106/

[21] "Data Exfiltration: Increasing Number of Tools Leveraged by Ransomware Attackers | Symantec Enterprise Blogs." Accessed: Sep. 16, 2024. [Online]. Available: https://symantec-enterprise-blogs.security.com/threat-intelligence/ransomware-data-exfiltration

[22] N. Scaife, H. Carter, P. Traynor, and K. R. B. Butler, "CryptoLock (and Drop It): Stopping Ransomware Attacks on User Data," *Proc Int Conf Distrib Comput Syst*, vol. 2016-August, pp. 303–312, Aug. 2016, doi: 10.1109/ICDCS.2016.46.

[23] Z. G. Chen, H. S. Kang, S. N. Yin, and S. R. Kim, "Automatic ransomware detection and analysis based on dynamic API calls flow graph," *Proceedings of the 2017 Research in Adaptive and Convergent Systems, RACS 2017*, vol. 2017-January, pp. 196–201, Sep. 2017, doi: 10.1145/3129676.3129704.

[24] "Execution, Tactic TA0002 - Enterprise | MITRE ATT&CK®." Accessed: Aug. 04, 2024. [Online]. Available: https://attack.mitre.org/tactics/TA0002/

[25] “Persistence, Tactic TA0003 - Enterprise | MITRE ATT&CK®.” Accessed: Aug. 04, 2024. [Online]. Available: https://attack.mitre.org/tactics/TA0003/

[26] “Boot or Logon Autostart Execution, Technique T1547 - Enterprise | MITRE ATT&CK®.” Accessed: Aug. 04, 2024. [Online]. Available: https://attack.mitre.org/techniques/T1547/

[27] “Defense Evasion, Tactic TA0005 - Enterprise | MITRE ATT&CK®.” Accessed: Aug. 10, 2024. [Online]. Available: https://attack.mitre.org/tactics/TA0005/

[28] N. Elgohary and N. Abdelbaki, “Detecting Mimikatz in Lateral Movements Using Windows API Call Sequence Analysis,” *NILES 2022 - 4th Novel Intelligent and Leading Emerging Sciences Conference, Proceedings*, pp. 306–310, 2022, doi: 10.1109/NILES56402.2022.9942377.

[29] T. Mcintosh, A. Bank, and A. Paul Watters, “Ransomware Reloaded: Re-examining Its Trend, Research and Mitigation in the Era of Data Exfiltration,” 2024, doi: 10.1145/3691340.

[30] “LOTS Project - Living Off Trusted Sites.” Accessed: Sep. 16, 2024. [Online]. Available: https://lots-project.com/

[31] “Windows System Programming - Johnson M. Hart - Google Books.” Accessed: Aug. 04, 2024. [Online]. Available: https://books.google.co.uk/books?hl=en&lr=&id=DYw5G55hJocC&oi=fnd&pg=PT23&dq=windows+system+programming+book&ots=I6gk0PTjSB&sig=HJDNNBBkDh6JZ6XTKfS_ROSmpSk&redir_esc=y#v=onepage&q=windows%20system%20programming%20book&f=false

[32] F. Cicala and E. Bertino, “Analysis of Encryption Key Generation in Modern Crypto Ransomware,” *IEEE Trans Dependable Secure Comput*, vol. 19, no. 2, pp. 1239–1253, 2022, doi: 10.1109/TDSC.2020.3005976.

[33] R. Moussaileb, N. Boulahia Cuppens, J.-L. Lanet, H. Le Bouder, and H. A. Le Bouder, “Survey On Windows-Based Ransomware Taxonomy And Detection Mechanisms: Case Closed,” vol. 2022, no. 6, pp. 1–36, doi: 10.1145/3453153ï.

[34] S. Mehnaz, A. Mudgerikar, and E. Bertino, “RWGuard: A Real-Time Detection System Against Cryptographic Ransomware,” *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 11050 LNCS, pp. 114–136, 2018, doi: 10.1007/978-3-030-00470-5_6.

[35] T. M. Lewis and B. P. Rimal, “Effects of Removing User-Land Hooks in Endpoint Protection During Attack Experiments,” *IEEE Access*, vol. 12, pp. 15820–15844, 2024, doi: 10.1109/ACCESS.2024.3357525.

[36] “Kernel Patch Protection - Wikipedia.” Accessed: Oct. 20, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Kernel_Patch_Protection

[37] L. Babun, H. Aksu, S. Uluagac, J. Lopez, · A Selcuk Uluagac, and A. S. Uluagac, “A Survey on Function and System Call Hooking Approaches,” *Article in Journal of Hardware and Systems Security*, 2017, doi: 10.1007/s41635-017-0013-2.

[38] Y. A. Ahmed, B. Koçer, S. Huda, B. A. Saleh Al-rimy, and M. M. Hassan, “A system call refinement-based enhanced Minimum Redundancy Maximum Relevance method for ransomware early detection,” *Journal of Network and Computer Applications*, vol. 167, p. 102753, Oct. 2020, doi: 10.1016/J.JNCA.2020.102753.

[39] “Windows Internals: System architecture, processes, threads, memory ... - Pavel Yosifovich, David A. Solomon, Mark E. Russinovich, Alex Ionescu - Google Books.” Accessed: Jul. 08, 2024. [Online]. Available: https://books.google.co.uk/books?hl=en&lr=&id=y83LDgAAQBAJ&oi=fnd&pg=PA1999&dq=windows+internals+seventh+edition&ots=kJ8pPuNjgz&sig=gO1huPJiCmpjQPWHwW-m0QIsVwM&redir_esc=y#v=onepage&q=windows%20internals%20seventh%20edition&f=false

[40] “(6) Intro to Syscalls for Windows Malware - YouTube.” Accessed: Sep. 08, 2024. [Online]. Available: https://www.youtube.com/watch?v=elA_eiqWefw

[41] S. Hussain *et al.*, “Towards Reproducible Ransomware Analysis,” 2023, doi: 10.1145/3607505.3607510.

[42] M. I. Al-Saleh and H. M. Hamdan, “On Studying the Antivirus Behavior on Kernel Activities,” 2018, doi: 10.1145/3230348.3230376.

[43] “OverviewInterception · microsoft/Detours Wiki · GitHub.” Accessed: Oct. 20, 2024. [Online]. Available: https://github.com/microsoft/Detours/wiki/OverviewInterception

[44] “Ransomware Detection Using Windows API Calls and Machine Learning.” Accessed: Sep. 29, 2024. [Online]. Available: https://vtechworks.lib.vt.edu/items/0cf2c98c-183c-4fce-a116-8f01ba3c5cd7

[45] L. Cizeron, “How and why do ransomwares escape the notice of some security tools?”, Accessed: Jun. 29, 2024. [Online]. Available: https://www.researchgate.net/publication/368872689

[46] S. Alzahrani, Y. Xiao, and W. Sun, “An Analysis of Conti Ransomware Leaked Source Codes,” *IEEE Access*, vol. 10, pp. 100178–100193, 2022, doi: 10.1109/ACCESS.2022.3207757.

[47] G. Bernardinetti, D. Di Cristofaro, and G. Bianchi, “Windows Antivirus Evasion Techniques: How to Stay Ahead of the Hooks,” 2023, Accessed: Sep. 29, 2024. [Online]. Available: https://www.trendmicro.com/vinfo/us/security/definition/indicators-of-compromise

[48] “3 EDR Bypass Techniques & How to Stop Them | Deep Instinct.” Accessed: Nov. 02, 2024. [Online]. Available: https://www.deepinstinct.com/blog/edr-bypass-techniques-and-how-to-stop-them

[49] “GitHub - j00ru/windows-syscalls: Windows System Call Tables (NT/2000/XP/2003/Vista/7/8/10/11).” Accessed: Nov. 02, 2024. [Online]. Available: https://github.com/j00ru/windows-syscalls

[50] “Direct Syscalls: A journey from high to low - RedOps - English.” Accessed: Nov. 01, 2024. [Online]. Available: https://redops.at/en/blog/direct-syscalls-a-journey-from-high-to-low

[51] “Direct Syscalls vs Indirect Syscalls - RedOps - English.” Accessed: Nov. 11, 2024. [Online]. Available: https://redops.at/en/blog/direct-syscalls-vs-indirect-syscalls

[52] M. N. Olaimat, M. Aizaini Maarof, and B. A. S. Al-Rimy, “Ransomware Anti-Analysis and Evasion Techniques: A Survey and Research Directions,” *2021 3rd International Cyber Resilience Conference, CRC 2021*, Jan. 2021, doi: 10.1109/CRC50527.2021.9392529.

[53] “PikaBot: a Guide to its Deep Secrets and Operations - Sekoia.io Blog.” Accessed: Oct. 25, 2024. [Online]. Available: https://blog.sekoia.io/pikabot-a-guide-to-its-deep-secrets-and-operations/

[54] “Best Endpoint Detection and Response Solutions Reviews 2024 | Gartner Peer Insights.” Accessed: Oct. 20, 2024. [Online]. Available: https://www.gartner.com/reviews/market/endpoint-detection-and-response-solutions

[55] “Research Methods for Cyber Security - Thomas W. Edgar, David O. Manz - Google Books.” Accessed: Aug. 10, 2024. [Online]. Available: https://books.google.co.uk/books?hl=en&lr=&id=aRI2DQAAQBAJ&oi=fnd&pg=PP1&dq=research+methods+for+cyber+security&ots=SmUPWBRxD_&sig=2KGTp6fK39Hv38ucerzRfHJ-9cY&redir_esc=y#v=onepage&q=research%20methods%20for%20cyber%20security&f=false

[56] “GitHub - mpasco/MalbehavD-V1: Public datasets of malware and benign executable files (Windows EXE files). The dataset can be used by cybersecurity researchers focusing on the area of malware detection. It is suitable for training and testing both machine learning and deep learning algorithms.” Accessed: Sep. 29, 2024. [Online]. Available: https://github.com/mpasco/MalbehavD-V1

Appendix

```
[+] Creating HookChain implants
0x00007FF7ED029210 = &SyscallList
0x00007FF9FB600000 = &Ntdll base
0x00007FF9FB600000 = &Ntdll real base
0x0000021DD3324040 = pImageExportDirectory
0x0000021DD3324040 = lpLocalAddress
[>] Hooked Ntdll Syscall List:
  |--> Entries[025] SSN = 0x0018, Address = 0x00007FF9FB6A4200: NtAllocateVirtualMemory
  |--> Entries[118] SSN = 0x004E, Address = 0x00007FF9FB6A48C0: NtCreateThread
  |--> Entries[119] SSN = 0x00C5, Address = 0x00007FF9FB6A5790: NtCreateThreadEx
  |--> Entries[127] SSN = 0x00CD, Address = 0x00007FF9FB6A5890: NtCreateUserProcess
  |--> Entries[134] SSN = 0x0034, Address = 0x00007FF9FB6A4580: NtDelayExecution
  |--> Entries[242] SSN = 0x0026, Address = 0x00007FF9FB6A43C0: NtOpenProcess
  |--> Entries[268] SSN = 0x0050, Address = 0x00007FF9FB6A4900: NtProtectVirtualMemory
  |--> Entries[325] SSN = 0x0023, Address = 0x00007FF9FB6A4360: NtQueryVirtualMemory
  |--> Entries[338] SSN = 0x003F, Address = 0x00007FF9FB6A46E0: NtReadVirtualMemory
  |--> Entries[366] SSN = 0x0052, Address = 0x00007FF9FB6A4940: NtResumeThread
  |--> Entries[478] SSN = 0x003A, Address = 0x00007FF9FB6A4640: NtWriteVirtualMemory
  +--> Mapped 480 functions

[>] IAT Hook of: kernel32 => 0x00007FF9FB080000
  |--> Implant kernel32 ntdll.dll->NtAllocateVirtualMemory 0x00007FF7ED0188F9 SSN 0x18
  |--> Implant kernel32 ntdll.dll->NtQueryVirtualMemory 0x00007FF7ED01A069 SSN 0x23
  |--> Implant kernel32 ntdll.dll->NtProtectVirtualMemory 0x00007FF7ED019BF5 SSN 0x50
  |--> Implant kernel32 ntdll.dll->NtReadVirtualMemory 0x00007FF7ED01A16D SSN 0x3F
  |--> Implant kernel32 api-ms-win-core-memory-l1-1-0.dll->ReadProcessMemory 0x00007FF7ED0119E7
  |--> Implant kernel32 api-ms-win-core-memory-l1-1-0.dll->VirtualProtectEx 0x00007FF7ED0116EF
  |--> Implant kernel32 api-ms-win-core-libraryloader-l1-2-0.dll->GetProcAddress 0x00007FF7ED011505
  +--> Hooked 7 function(s)

[>] IAT Hook of: kernelbase => 0x00007FF9F9110000
  |--> Implant kernelbase ntdll.dll->TpSetTimer 0x00007FF7ED01A825 SSN 0x62
  |--> Implant kernelbase ntdll.dll->NtQueryVirtualMemory 0x00007FF7ED01A069 SSN 0x23
  |--> Implant kernelbase ntdll.dll->NtReadVirtualMemory 0x00007FF7ED01A16D SSN 0x3F
  |--> Implant kernelbase ntdll.dll->NtProtectVirtualMemory 0x00007FF7ED019BF5 SSN 0x50
  |--> Implant kernelbase ntdll.dll->NtWriteVirtualMemory 0x00007FF7ED01AC5D SSN 0x3A
  |--> Implant kernelbase ntdll.dll->NtAllocateVirtualMemory 0x00007FF7ED0188F9 SSN 0x18
  |--> Implant kernelbase ntdll.dll->NtDelayExecution 0x00007FF7ED01917D SSN 0x34
  |--> Implant kernelbase ntdll.dll->NtOpenProcess 0x00007FF7ED0199ED SSN 0x26
  |--> Implant kernelbase ntdll.dll->NtCreateThreadEx 0x00007FF7ED019051 SSN 0xC5
```